CEH v13 AI | Exam 312-50

Module 18
IoT and OT Hacking

Complete Exam-Focused Study Guide

Based on 39 supplied note pages
Includes commands, tools, practice questions, and flashcards

Table of Contents

1. Module at a Glance 5
2. Big Picture Map 6
3. Topic-by-Topic Guide 7
    3.1 Learning objectives, IoT meaning, and data flow 7
    3.2 IoT architecture, technologies, operating systems, and protocols 8
    3.3 OWASP IoT Top 10, attack surfaces, and device vulnerabilities 9
    3.4 IoT threat catalogue, DDoS botnets, and HVAC pivoting 10
    3.5 Rolling-code and jamming attacks 11
    3.6 Backdoors, Telnet, BlueBorne, and smart-grid compromise 12
    3.7 SDR attacks, local discovery, and DNS rebinding 13
    3.8 Fault injection and the remaining IoT attack types 14
    3.9 IoT malware and botnet families 15
    3.10 Information gathering with Shodan and wireless sniffing 16
    3.11 IoT vulnerability scanning and validation 17
    3.12 RFCrack, Open Sniffer, and HackRF One 18
    3.13 HackRF replay and hardware communication buses 18
    3.14 NAND glitching, CamOver, and Telnet access 19
    3.15 Maintaining access through firmware modification 20
    3.16 Firmware analysis and reverse engineering 21
    3.17 General IoT hacking and device-management tools 21
    3.18 IoT countermeasures, manufacturer guidance, and hardware practice 22
    3.19 OT, ICS components, and IT/OT convergence 23
    3.20 Purdue model and OT protocol placement 24
    3.21 OT challenges and vulnerabilities 25
    3.22 MITRE ATT&CK for ICS 26
    3.23 OT threats, HMI attacks, and side channels 27
    3.24 PLC rootkits, Evil PLC, RF controllers, and supply chains 28
    3.25 OT malware: Fuxnet and related families 29
    3.26 OT information gathering with Shodan and CIRT.net 30
    3.27 OT protocol discovery with Nmap, Wireshark, and Malcolm 31
    3.28 Nessus, Fuzzowski, and Skybox 31
    3.29 ICS hardware analysis, Metasploit, modbus-cli, and DNP3 32
    3.30 OT defenses, Purdue controls, zero trust, and organizations 33
4. Tool Encyclopedia 35
    4.1 Exposure search and information gathering 35

4.2 Discovery, scanning, analysis, and prioritization 35
4.3 Radio, hardware, and exploitation tools 36
4.4 Firmware and reverse-engineering tools 37
4.5 Device management and security products 38
5. Command Sheet 40
5.1 Shodan filters and HTTP discovery with Nmap 40
5.2 ifconfig, Airmon-ng, Airodump-ng, and Wireshark 40
5.3 IoTSeeker 41
5.4 RFCrack 42
5.5 HackRF transfer and device information 42
5.6 BUS Auditor UART and I2C scans 43
5.7 CamOver 44
5.8 Firmware Mod Kit extraction 44
5.9 Static firmware commands and QEMU emulation 45
5.10 Nmap industrial-protocol scripts 46
5.11 Fuzzowski protocol fuzzing 47
5.12 Metasploit Modbus auxiliary modules 48
5.13 modbus-cli 49
5.14 Reading the Fuxnet console evidence 50
6. Ports, Protocols, Standards and Acronyms 51
6.1 Core network and OT ports 51
6.2 Wireless, link, fieldbus, and industrial technologies 53
6.3 Standards and architecture terms 55
6.4 Quick acronym decoder 56
7. Attack vs Countermeasure and Look-alike Comparisons 58
7.1 Attacks and best-matched countermeasures 58
7.2 Look-alike pairs 59
8. Exam Cheat Sheet 61
8.1 Keyword -> answer 61
8.2 Must-memorize lists and numbers 62
8.3 Pick the BEST option rules 62
9. Practice Questions 64
Question 1: IoT architecture 64
Question 2: Rolling-code scenario 64
Question 3: Best first OT step 64
Question 4: Nmap flag reading 64

Question 5: OT script and protocol 65
Question 6: Firmware analysis 65
Question 7: Hardware interface 65
Question 8: RFCrack flag 65
Question 9: Purdue placement 65
Question 10: MITRE ATT&CK for ICS 66
Question 11: Evil PLC 66
Question 12: AI-assisted assessment 66
Question 13: Modbus command reading 66
Question 14: Metasploit module selection 67
Question 15: HackRF flags 67
Question 16: Default-credential scenario 67
Question 17: Firmware persistence 67
Question 18: Fuzzing tool 68
Question 19: OT malware recognition 68
Question 20: Segmentation scenario 68
Question 21: AI attack angle 68
Question 22: Supply-chain recognition 69
Question 23: Command output interpretation 69
Question 24: Best vulnerability-scanning decision 69
10. Flashcards 70
11. Glossary 71
12. Coverage Report 74
12.1 Page-to-section map 74
12.2 Unclear items and best readings 75
12.3 Items to double check 76
12.4 Coverage confirmation 76

Exam-focused study guide for CEH 312-50. Based on pages 1 to 39 of the supplied notes. Added material is marked ADDED CONTEXT.

1. Module at a Glance

Module 18: IoT and OT Hacking

- IoT connects physical objects to IP networks so they can sense and share data.
- OT monitors or changes physical industrial processes through ICS equipment.
- IoT attacks target devices, radios, firmware, cloud services, and default credentials.
- OT attacks target PLCs, HMIs, SCADA, industrial protocols, and trusted suppliers.
- Defense combines asset visibility, segmentation, secure updates, and safe monitoring.

Why it matters: A compromised camera can become a botnet member. A compromised PLC can stop production or damage equipment. Confidentiality matters, but OT also depends heavily on availability, integrity, safety, and deterministic timing.

What the exam usually asks: Purdue levels, ICS component roles, protocol-to-port matches, Modbus weakness, Nmap OT scripts, Shodan filters, SDR and firmware tools, rolling-code steps, OWASP IoT items, MITRE ATT&CK for ICS tactics, and the safest countermeasure for a live plant.

Ten things to remember most:

1. Edge -> access gateway -> internet -> middleware -> application is the IoT stack.
2. Purdue Level 0 is the physical process; Level 3 is site operations; Level 3.5 is the industrial DMZ; Levels 4 and 5 are enterprise systems.
3. PLC executes control logic; RTU collects remote field data; HMI displays and accepts operator commands; SCADA supervises; DCS controls a distributed plant.
4. Modbus TCP uses port 502 and normally has no authentication or encryption.
5. DNP3 uses 20000; EtherNet/IP uses 44818; S7 uses 102; BACnet uses UDP 47808.
6. A rolling-code attack jams reception, captures codes, forwards one, and saves another unused valid code for later.
7. Shodan finds exposed devices; Nmap identifies services; Nessus finds known vulnerabilities; Malcolm analyzes network traffic.
8. UART, SPI, I2C, and JTAG are hardware interfaces. JTAG is a debug/test interface.
9. Active scanning can crash fragile OT devices. Prefer passive discovery first.
10. Zero trust means verify explicitly and grant least privilege, even inside OT.

2. Big Picture Map

```
                IoT path
Sensors -> Edge -> Gateway -> Internet -> Middleware -> App
   |          |         |                    |
hardware    radio    exposure              APIs/data
   |          |         |                    |
JTAG/UART  replay   Shodan/Telnet       cloud privacy

                OT path
Enterprise L5/L4 -> IDMZ L3.5 -> Operations L3
                                      |
                               Supervisory L2
                                      |
                               Control L1 -> Process L0

Assessment: passive discovery -> safe validation -> report
Defense: inventory -> segment -> harden -> monitor -> recover
```

Area	Main target	Typical evidence	Best first defense
IoT	Device and ecosystem	Banner, firmware, radio	Unique credentials and updates
OT	Physical process	Passive traffic, asset data	Zones, conduits, safe monitoring
Both	Availability and trust	Logs and configuration	Least privilege and recovery

3. Topic-by-Topic Guide

3.1 Learning objectives, IoT meaning, and data flow

What it is: The Internet of Things (IoT), also called the Internet of Everything (IoE) in the notes, is a network of IP-addressable objects with embedded sensors, processors, and communication hardware. A "thing" can be natural, human-made, or machine-made. The module objectives are IoT concepts, attacks, methodology and defense, followed by the same four areas for OT.

Analogy: A smart thermometer is a receptionist who reports room temperature to a head office and accepts instructions back.

How it works:

1. Industrial, wearable, healthcare, and home devices sense or collect data.
2. They send data, often through an IoT gateway, across the internet.
3. Cloud storage or servers store and analyze the data.
4. A mobile app displays results and can send a remote-control command.
5. The gateway returns the command to the device, which changes its behavior.

Real-world use: A factory motor reports vibration. Analytics detects an unusual pattern, and an authorized maintenance application schedules inspection. If default credentials protect the gateway, an outsider could instead read or alter the data.

Tools and commands used here: No tool is needed to define the flow. Later pages use Shodan, Nmap, Wireshark, firmware tools, and radio tools against its weak points.

AI angle (ADDED CONTEXT): AI can classify sensor anomalies and summarize asset inventory. An attacker can use AI to prioritize exposed devices from public banners. Example defensive prompt: "Group these authorized asset records by protocol, owner, patch status, and internet exposure. Do not suggest active tests."

Countermeasures and defense: Know every data producer and consumer. Authenticate devices and users, encrypt transport, restrict commands, and log administrative changes.

> **EXAM FOCUS:** IoT is not only smart homes. The key property is embedded sensing or control plus network communication. A gateway translates or connects devices; it is not the same as cloud middleware. Trap: assuming every device directly uses a full TCP/IP stack and talks directly to an app.

Memory trick: "Sense, Send, Store, Study, Steer" describes sense, transmit, store, analyze, and command.

3.2 IoT architecture, technologies, operating systems, and protocols

What it is: The five layers separate physical sensing from user applications.

Analogy: A parcel moves from a local shop, through a regional depot and roads, to a sorting warehouse, and finally to the customer.

How it works:

1. **Edge technology:** sensors, devices, machines, and intelligent edge nodes.
2. **Access gateway:** protocol translation and messaging between local and IP networks.
3. **Internet:** connectivity between endpoints and remote services.
4. **Middleware:** device management and information management.
5. **Application:** delivers applications to different IoT users.

Technology groups from the notes:

- Short range: BLE, Li-Fi, NFC, QR codes and barcodes, RFID, Thread, Wi-Fi, Wi-Fi Direct, Z-Wave, Zigbee, and ANT.
- Medium range: HaLow and LTE-Advanced.
- Long range: LPWAN, LoRaWAN, 6LoWPAN, Sigfox, Neul, VSAT, cellular, and NB-IoT.
- Wired: Ethernet, MoCA, and power-line communication (PLC), which is unrelated to a programmable logic controller with the same abbreviation.
- Operating systems: Windows 10 IoT, Amazon FreeRTOS, Fuchsia, RIOT, Ubuntu Core, ARM Mbed OS, Zephyr, Embedded Linux, NuttX RTOS, Integrity RTOS, and Tizen.
- Application protocols: CoAP, Edge, LWM2M, Physical Web, XMPP, Mihini/M3DA, and MQTT.

Real-world use: A battery soil sensor can use Zigbee to a gateway. The gateway uses IP networking to reach middleware, where MQTT messages become application data.

Tools and commands used here: Wireshark can decode MQTT and Zigbee. Open Sniffer, CatSniffer, and KillerBee examine low-power wireless traffic.

> **ADDED CONTEXT:** MQTT is lightweight publish/subscribe messaging through a broker. CoAP is a REST-like protocol for constrained devices. LWM2M adds device management. XMPP is extensible XML messaging. 6LoWPAN adapts IPv6 to IEEE 802.15.4 links; it is not simply a long-range radio like LoRaWAN.

AI angle (ADDED CONTEXT): A model can explain a decoded protocol or flag an unexpected topic. Example prompt: "Explain this authorized MQTT packet capture field by field and identify plaintext metadata without inventing payload contents."

Countermeasures and defense: Authenticate the gateway and broker, restrict topics, encrypt where supported, and remove unused radio and management services.

> **EXAM FOCUS:** Match the function to the layer. Protocol translation means access gateway; device management means middleware; sensors mean edge. Trap: placing MQTT at the physical layer. It is an application protocol, normally carried over TCP.

Memory trick: "Every Access Internet Message Arrives": Edge, Access, Internet, Middleware, Application.

3.3 OWASP IoT Top 10, attack surfaces, and device vulnerabilities

What it is: OWASP lists recurring weaknesses, not a sequence of attacks.

Analogy: A building risk register lists unlocked doors, copied keys, old cameras, and unguarded loading bays before describing individual burglaries.

OWASP Top 10 IoT threats in notes order:

1. Weak, guessable, or hardcoded passwords.
2. Insecure network services.
3. Insecure ecosystem interfaces.
4. Lack of secure update mechanisms.
5. Use of insecure or outdated components.
6. Insufficient privacy protection.
7. Insecure data transfer and storage.
8. Lack of device management.
9. Insecure default settings.
10. Lack of physical hardening.

Eighteen attack surfaces: ecosystem, device memory, physical interfaces, web interface, firmware, network services, administrative interface, local storage, cloud web interface, third-party backend APIs, update mechanism, mobile application, vendor backend APIs, ecosystem communication, network traffic, authentication/ authorization, privacy, and hardware sensors.

The seventeen vulnerability lessons:

- Username enumeration reveals valid names through different authentication responses.
- Weak passwords such as 1234 and unchanged factory passwords permit guessing.
- No lockout after three to five failures permits continued guessing.

- Unencrypted services expose traffic; weak implementations such as SSLv2 are not adequate merely because encryption is present.
- Missing two-factor authentication leaves one stolen secret sufficient.
- Updates without TLS or file encryption, and world-writable update storage, allow tampering. No manual check or no update mechanism leaves flaws unresolved.
- Denial of service can disable one service or the entire device.
- Removable storage and firmware extraction can reveal source, binaries, passwords, and SSH keys. Hidden firmware version and update date hinder risk decisions.
- JTAG with a debugger can alter execution; side channels can leak or change behavior.
- Serial console access can yield a full shell even when boot protections exist.
- Outdated BusyBox, OpenSSL, SSH, or web server components carry known weaknesses.

Real-world use: An authorized reviewer extracts a camera firmware image, finds a hardcoded credential, and reports it before the same credential is used at scale.

Tools and commands used here: Binwalk, Firmware Mod Kit, strings, hexdump, GDB, OpenOCD, BUS Auditor, Shodan, and Nmap appear in later topics.

AI angle (ADDED CONTEXT): AI can map findings to OWASP categories, but evidence must support each mapping. Prompt: "Classify these verified findings under the OWASP IoT Top 10 and quote the supporting observation for each."

Countermeasures and defense: Unique credentials, lockout, MFA, signed updates, component inventory, data minimization, encrypted storage, lifecycle management, secure defaults, disabled debug ports, and tamper resistance address the list.

> **EXAM FOCUS:** An insecure mobile app, cloud API, or backend belongs to the IoT ecosystem too. Trap: choosing only the physical device. Hardcoded password is a credential flaw; outdated OpenSSL is an insecure/outdated component.

Memory trick: For the Top 10, group them as "Secrets, Services, Surrounding interfaces, Software updates and Supply components; Privacy, Protection of data, Provisioning, Presets, Physical access."

3.4 IoT threat catalogue, DDoS botnets, and HVAC pivoting

What it is: The notes list 21 threats: DDoS, HVAC attack, rolling code, BlueBorne, jamming, backdoor remote access, Telnet remote access, Sybil, exploit kits, MITM, replay, forged malicious device, side channel, ransomware, client impersonation, SQL injection, SDR-based attack, fault injection, network pivoting, DNS rebinding, and firmware-over-the-air (FOTA) update attack.

Analogy: A botnet is a crowd of compromised smart devices all knocking on one door at the attacker's signal.

How an IoT DDoS works:

1. The attacker exploits device weaknesses and installs malicious software.
2. Remote access turns many devices into bots, collectively an army of botnets.
3. A command-and-control (C2 or C&C) center sends instructions.
4. Distributed bots send a large request volume to the target.
5. The server exhausts resources, goes offline, or cannot process further requests.

HVAC attack steps:

1. Shodan identifies an exposed, vulnerable industrial control or HVAC system.
2. Default credentials provide access to the ICS/HVAC interface.
3. The attacker changes temperature or uses the trusted connection to enter the LAN.
4. The compromised building system becomes a network-pivoting foothold.

Real-world use: An assessor finds that a facilities network can reach finance. The safest result is a segmentation finding, not an attempt to alter temperature.

Tools and commands used here: Shodan, Censys, Nmap, botnet malware, and the iRidium interface shown in the HVAC example.

AI angle (ADDED CONTEXT): Attackers can generate convincing phishing content for initial compromise. Defenders can cluster traffic to identify bot-like behavior. Prompt: "Summarize common timing and destination features in this authorized flow export and list benign explanations."

Countermeasures and defense: Remove internet exposure, change credentials, patch, segment facilities networks, rate-limit, filter egress, and prepare DDoS absorption with the ISP or upstream provider.

> **EXAM FOCUS:** C2 coordinates bots; the target server is the victim of the flood. HVAC is often the initial access or pivot, not the final target. Trap: calling every large infection a DDoS before a flood is launched.

Memory trick: DDoS chain is "Break in, Bot, Beacon, Blast, Blackout."

3.5 Rolling-code and jamming attacks

What it is: A rolling code, or hopping code, changes on each key-fob use to stop simple reuse. The described attack combines jamming, capture, and selective replay. Jamming means transmitting interference on the same frequency so endpoints cannot reliably send or receive messages.

Analogy: An attacker blocks a mailbox, collects two signed entry slips, hands over the first to avoid suspicion, and keeps the second unused slip.

How the rolling-code attack works:

1. The victim presses the fob. The attacker jams the car receiver and sniffs code one.
2. The car does not unlock, so the victim presses again.
3. The attacker captures code two and forwards code one, which unlocks the car.
4. Code two remains unused and valid in the receiver's expected sequence.
5. The attacker later replays code two to unlock the vehicle.

How jamming works: The attacker determines the sensor frequency, transmits random radio energy there, and reduces the signal-to-noise ratio until communication fails.

Real-world use: A vehicle-security team tests a owned fleet receiver in a shielded environment and confirms that request failure plus a second press is a warning sign.

Tools and commands used here: RFCrack, YardStick One, HackRF One, RTL-SDR, `RFCrack.py -j`, and `RFCrack.py -r` with modulation and frequency options.

Countermeasures and defense: Use authenticated, freshness-protected protocols, receiver-side anomaly detection, rolling-window limits, and user warnings after repeated failed unlocks. Encryption alone does not prevent radio jamming.

> **EXAM FOCUS:** The second code is saved; the first code is forwarded to preserve normal behavior. Trap: a basic replay resends an old message, while this scenario specifically needs jamming plus capture of an unused future code.

Memory trick: "Jam one, Grab one; Jam two, Grab two; Send one, Save two."

3.6 Backdoors, Telnet, BlueBorne, and smart-grid compromise

What it is: A backdoor is hidden or unauthorized access that bypasses normal authentication. Telnet is an old remote terminal protocol whose normal traffic, including credentials, is plaintext. BlueBorne is a family of Bluetooth attacks; the notes associate it with airborne replay, fuzzing, and jamming using SDR hardware.

Analogy: A phishing attachment is a fake delivery worker who quietly installs a spare key inside the building.

How the smart-grid example works:

1. The attacker profiles employees using social engineering.
2. A legitimate-looking phishing email carries a malicious attachment.
3. Opening it installs a backdoor and gives access to the private network.
4. The attacker pivots to SCADA and disables a UPS or other power controls.

5. Loss of power causes operational impact.

How Telnet access works: Scanning or Shodan reveals TCP 23. Weak/default login details or an unprotected service then provide a remote command session. The example also shows a Dahua DVR and HTTP port 80, so an open web port is not itself Telnet.

Real-world use: A utility removes direct Telnet, requires a VPN and jump host, and trains staff to report attachments instead of testing them on production systems.

Tools and commands used here: Shodan, Censys, Nmap, HackRF One, and Telnet.

AI angle (ADDED CONTEXT): Generative AI can make phishing grammatically convincing and deepfake voice can support impersonation. Defenders should verify unusual requests through a second channel. Prompt for awareness work: "List benign warning signs in this fictional email without improving its persuasive content."

Countermeasures and defense: Use SSH rather than Telnet, email filtering, attachment sandboxing, MFA, least privilege, Bluetooth patching, and restricted device discovery.

> **EXAM FOCUS:** SSH is the secure Telnet replacement. BlueBorne is Bluetooth, not Wi-Fi. Trap: choosing a firewall as the complete answer to phishing; user reporting and endpoint/email controls address the initial attachment.

Memory trick: "Telnet Talks in Text"; "BlueBorne Bites Bluetooth."

3.7 SDR attacks, local discovery, and DNS rebinding

What it is: Software-defined radio (SDR) implements radio functions in software. It can inspect, modify, generate, or replay signals. DNS rebinding makes a browser resolve one hostname first to an attacker and then to a private address, helping malicious script reach local devices through the victim browser.

Analogy: SDR is a programmable musical instrument that can learn and replay many songs. DNS rebinding is a signpost that is changed after a visitor passes the gate.

Three SDR methods in the notes:

1. **Replay:** discover the frequency, capture an original command, isolate the command sequence, and inject it back into the IoT network.
2. **Cryptanalysis:** capture and reverse-engineer the signal/protocol, requiring knowledge of cryptography, communication theory, and modulation.
3. **Reconnaissance:** inspect specifications and hardware; a multimeter and chipset inspection can identify ground pins, product IDs, and other information.

Local device discovery steps:

1. A user visits an attacker-controlled advertisement or page.

2. Malicious code obtains the local IP and probes available private addresses.
3. Active devices may return TCP RST; inactive ones may time out.
4. Responses reveal devices and potentially access-point UID and BSSID information.
5. A tool such as Singularity of Origin tests DNS rebinding.
6. Successful rebinding lets the page interact with local device interfaces.

Real-world use: A browser-security assessment uses a dedicated test network to show why an unauthenticated local camera interface is reachable from a web page.

Tools and commands used here: HackRF One, RTL-SDR, RFCrack, Singularity of Origin, Wireshark, and TCP SYN/RST responses.

Countermeasures and defense: Authenticate local interfaces, validate Host headers, use DNS rebinding protections, isolate IoT networks, and require explicit user approval for local network access where the platform supports it.

> **EXAM FOCUS:** RST can reveal a closed port on a live host; timeout gives less certain evidence. BSSID identifies a wireless access point radio. Trap: calling rebinding ordinary DNS spoofing; rebinding abuses changing answers and browser trust.

Memory trick: SDR does "Record, Reverse, Recon"; rebinding does "Answer, Alter, Access."

3.8 Fault injection and the remaining IoT attack types

What it is: Fault injection, also called perturbation, deliberately disturbs hardware so it skips checks, leaks data, or executes unintended behavior. It can be invasive, which alters or opens the package, or non-invasive, which works externally.

Analogy: A momentary flicker can make a cashier skip a verification step.

Fault methods and other attacks:

- Optical, electromagnetic fault injection (EMFI), and body-bias injection (BBI) use lasers, electromagnetic pulses, or bias changes.
- Power, clock, and reset glitching disturb supply or timing.
- Voltage/frequency tampering changes operating conditions; temperature attacks alter the chip environment.
- Sybil uses multiple forged identities, for example to simulate traffic congestion.
- Exploit kits automate scripts against poorly patched vulnerabilities.
- MITM secretly intercepts and can modify two-party communication.
- Replay resends captured valid messages, potentially causing repeated actions or DoS.
- Forged malicious device replacement needs physical or supply-chain access.

- Side channel derives secrets from emissions or behavior rather than breaking math.
- Ransomware locks a screen or encrypts files to deny access until payment is demanded.
- Client impersonation pretends to be a legitimate client; SQL injection inserts database commands through unsafe input; FOTA attacks abuse firmware updates.

Real-world use: A hardware lab tests whether a voltage glitch bypasses secure boot on a development board, then adds brownout detection and boot verification.

Tools and commands used here: Oscilloscope, signal analyzer, multimeter, glitching equipment, and debug interfaces.

Countermeasures and defense: Use secure boot, signed firmware, glitch sensors, constant-time authentication, tamper evidence, nonces and timestamps for anti-replay, input validation, parameterized SQL, and offline backups.

> **EXAM FOCUS:** Fault injection changes execution conditions; side channel observes behavior. Sybil means many identities; MITM means an intermediary. Trap: selecting encryption as an anti-jamming control or a replay defense without freshness.

Memory trick: "Light, Magnet, Bias; Power, Clock, Reset; Volt, Hertz, Heat."

3.9 IoT malware and botnet families

What it is: IoT malware turns weakly protected devices into botnet members, proxies, miners, ransomware victims, or attack platforms.

Analogy: One unchanged factory key can open thousands of houses of the same model.

How it works: KmsdBot's Kmsdx variant adds Telnet scanning/authentication, targets SSH exposure and default credentials, and supports more CPU architectures. The notes' Go-source analysis mentions `redress`, `main.go`, `ssh.go`, `telnet.go`, and a `telnet.txt` credential list. These are static-analysis clues, not commands to run. Other listed families are WailingCrab, P2PInfect, NKAbuse, IoTroop, and XorDdos.

> **ADDED CONTEXT:** IoTroop is also known as Reaper. XorDdos targets Linux systems. P2PInfect uses peer-to-peer propagation and has targeted Redis. NKAbuse uses NKN networking for decentralized communication. WailingCrab has used MQTT for C2. Family capabilities change, so answer from the behavior described in the question.

Real-world use: A defender finds repeated outbound Telnet attempts from a camera. The response is isolation, credential replacement, firmware remediation, and checking peer devices rather than merely deleting one suspicious process.

Tools and commands used here: Redress for Go binary inspection, Shodan, Censys, network monitoring, and anti-malware tools.

AI angle (ADDED CONTEXT): AI can help label static functions but may hallucinate malware capabilities. Prompt: "Describe only observable behavior in these decompiled snippets and mark uncertain claims; do not generate propagation code."

Countermeasures and defense: Unique credentials, disable Telnet, restrict egress, patch firmware, monitor unusual scanning, and rebuild compromised devices from trusted images.

> **EXAM FOCUS:** Default credentials plus broad architecture support increase botnet reach. Trap: treating Telnet scanning as exploitation of SSH; they are different services even when one malware family targets both.

Memory trick: "Scan, Sign in, Spread, Serve C2."

3.10 Information gathering with Shodan and wireless sniffing

What it is: Information gathering identifies exposed devices and their protocols before vulnerability validation. Shodan indexes internet-facing services; sniffing observes traffic within range or on a monitored network.

Analogy: Shodan is a searchable directory of public shop signs; sniffing is listening to conversations on a network you are authorized to monitor.

How it works:

1. Query Shodan for `webcamxp country:"US"`, `webcamxp city:"paris"`, or `webcamxp geo:"-50.81,201.80"` as shown in the notes.
2. Review IP, hostname, ISP, location, ports, and service banners without assuming that a banner proves exploitability.
3. Scan authorized ranges for HTTP ports 80, 81, 8080, and 8081.
4. Identify the wireless adapter with `ifconfig`, enable monitor mode, discover the target channel, then capture that channel in Wireshark.
5. The notes' capture shows MQTT Ping Request/Response and Publish exchanges, with destination TCP 1883.

Real-world use: An asset owner confirms that no camera is internet exposed and uses an approved wireless capture to identify plaintext telemetry.

Tools and commands used here: Shodan, MultiPing, FCC ID Search, Censys, FOFA, Nmap, ifconfig, Airmon-ng, Airodump-ng, Wireshark, Suphacap, and IoT sniffing tools.

AI angle (ADDED CONTEXT): ChatGPT or ShellGPT can explain a command before an approved scan. Prompt: "Explain each Nmap flag in this authorized HTTP discovery command and warn which options increase traffic." Never paste private scope or credentials into a public model.

Countermeasures and defense: Remove public administration interfaces, change identifying banners where practical, encrypt MQTT with TLS, use broker ACLs, and monitor for unexpected monitor-mode or scanning activity.

> **EXAM FOCUS:** Shodan is search, not an exploit framework. Monitor mode captures wireless frames; the channel must match. Trap: `airodump-ng` scans/captures while `airmon-ng` configures monitor mode.

Memory trick: "Shodan Sees, Nmap Scans, Wireshark Watches."

3.11 IoT vulnerability scanning and validation

What it is: Vulnerability scanning checks discovered devices for default credentials, known weaknesses, product paths, and versions.

Analogy: A safety inspector checks each lock model against a recall list instead of forcing every door.

How it works: IoTSeeker's Perl scanner accepts IP addresses and reports device type and whether the default password remains. Genzai identifies routers, cameras, and HMIs, then tests default passwords and vulnerabilities based on paths and versions. Its example mentions a TP-Link router result. Other listed tools are beSTORM, IoTSploit, IoTVAS, and Firmalyzer.

Real-world use: A hospital runs an approved credentialed scan against decommissioned test cameras and prioritizes any device still using factory credentials.

Tools and commands used here: IoTSeeker/IotScanner.pl, Genzai, beSTORM, IoTSploit, IoTVAS, Firmalyzer, and Nmap.

Countermeasures and defense: Use an asset inventory, authenticated configuration review, vendor advisories, firmware updates, and manual validation of high-risk results. Avoid password attempts or intrusive scans outside authorization and maintenance windows.

> **EXAM FOCUS:** Discovery finds devices; vulnerability scanning maps them to known weaknesses; exploitation proves impact. Trap: treating "changed password" as proof that the password is strong or the device is fully patched.

Memory trick: "Seeker Seeks Secrets; Genzai Gathers Versions."

3.12 RFCrack, Open Sniffer, and HackRF One

What it is: These are radio test tools. RFCrack works with YardStick One for sub-GHz tests. Open Sniffer captures IEEE 802.15.4/Zigbee frames into Wireshark and can transmit frames. HackRF One is a half-duplex SDR covering about 1 MHz to 6 GHz.

Analogy: They are universal radio microscopes, with some models also able to speak.

How it works:

1. RFCrack live replay uses `-i`; rolling-code mode uses `-r` with `-M MOD_2FSK` and a frequency such as `-F 314350000`; `-U -100 -L -10` sets RSSI bounds; `-j -F 314000000` selects jamming.
2. Open Sniffer captures Zigbee beacon and command frames, permits visualization, supports user-defined replay, and can continuously emit packets for DoS testing.
3. HackRF supports receive and transmit, but not simultaneously in full duplex. It can inspect protocols from GSM to Z-Wave and support replay, fuzzing, or jamming.

Real-world use: A lab uses RFCrack on a owned sub-GHz gate controller and Open Sniffer on an isolated Zigbee network to verify replay resistance.

Tools and commands used here: RFCrack.py, YardStick One, Open Sniffer, Wireshark, HackRF One, KillerBee, and CatSniffer.

Countermeasures and defense: Use cryptographic authentication and freshness, monitor spectrum anomalies, reduce radio range where possible, and test only in a controlled RF environment.

> **EXAM FOCUS:** MOD_2FSK is modulation, frequency is in Hz, RSSI bounds select signal strength, and half duplex means one direction at a time. Trap: selecting Wireshark as the radio transmitter; it primarily decodes and analyzes captures.

Memory trick: "I is Immediate replay, R is Rolling, J is Jamming, F is Frequency."

3.13 HackRF replay and hardware communication buses

What it is: A replay capture stores radio samples and transmits them again. Hardware buses expose internal communications and debugging functions.

Analogy: A sample file is an audio recording of a radio conversation. UART, SPI, I2C, and JTAG are different service hatches on the circuit board.

How it works:

1. FCC ID records or an RTL-SDR help identify a target transmitter's frequency.
2. `hackrf_transfer -r` receives raw samples; `-t` transmits a capture; `-f` sets frequency, and gain options control receive or transmit amplification.

3. `hackrf_info` confirms board identity and firmware version.
4. BUS Auditor scans pin combinations and voltage settings for UART or I2C.
5. UART output can reveal RX/TX pins and baud rate such as 9600; I2C output reveals device address plus SCL clock and SDA data pins.

Interface meanings: UART is asynchronous serial communication. SPI is synchronous serial communication with a controller and peripherals. I2C is addressed two-wire communication. JTAG is a test/debug interface useful for firmware and execution access. DIVA is a training application, and a PCB is the physical circuit board inspected.

Real-world use: A product team locates an enabled debug UART on a prototype, observes boot logs, and disables sensitive console access before release.

Tools and commands used here: HackRF One, RTL-SDR, BUS Auditor, DIVA, multimeter, GDB, and OpenOCD.

Countermeasures and defense: Lock or disable debug interfaces in production, require authenticated debug unlock, use tamper detection, and never rely on hidden test points as a security boundary.

> **EXAM FOCUS:** UART commonly leads to a console; JTAG commonly leads to debugging; I2C uses SCL and SDA. Trap: confusing a baud rate with radio frequency.

Memory trick: "UART Usually A Root Terminal; JTAG Joins Test And Debug Gates."

3.14 NAND glitching, CamOver, and Telnet access

What it is: NAND glitching interrupts boot by grounding a serial I/O pin of flash memory, with the goal of bypassing normal startup and gaining root. CamOver targets certain network-camera password-disclosure weaknesses.

Analogy: NAND glitching distracts a guard during the first seconds of a shift. CamOver uses a known defective desk procedure to ask for the administrator key.

How it works:

1. For NAND glitching, identify the flash and timing, disturb serial I/O during boot, and inspect whether a privileged console appears. Results are hardware dependent.
2. Start CamOver with `camover`; choose one address with `-a`, input files with `-i`, output with `-o`, threads with `-t`, or internet results through Shodan/ZoomEye.
3. For Telnet, identify open TCP 23 and use only authorized credentials. Shodan or Censys may expose a device banner before any login attempt.

Real-world use: An authorized camera audit verifies whether a patched firmware release removes password disclosure and whether Telnet is fully disabled.

Tools and commands used here: CamOver, Shodan, ZoomEye, Censys, Nmap, Telnet, multimeter, and hardware-glitching tools.

Countermeasures and defense: Patch camera firmware, restrict management networks, replace Telnet with SSH, require strong unique credentials, and add boot/flash glitch detection.

> **EXAM FOCUS:** CamOver is a camera exploitation tool, not a generic camera search engine. `-a` is one address; `-t` means threads in its usage, not target. Trap: trying Shodan API access without a key when a local `-a` test is intended.

Memory trick: "Camera Address = CamOver -a; Telnet Twenty-Three = Text."

3.15 Maintaining access through firmware modification

What it is: Firmware is the low-level software that runs a device. An attacker who modifies and reinstalls it can persist across ordinary account changes.

Analogy: Instead of keeping a copied house key, the intruder replaces the lock's internal instructions so it always admits them.

How it works:

1. After authorized access in an assessment, obtain the firmware image.
2. Firmware Mod Kit's `extract-firmware.sh` identifies and extracts components.
3. The DD-WRT example shows a TRX header, gzip/LZMA compressed data, and a Squashfs filesystem, plus a checksum and offsets.
4. Modify files only in an owned lab image, reconstruct it, and verify whether the device accepts unsigned or improperly validated firmware.
5. The security finding is an update-verification failure with persistence impact.

Real-world use: A vendor test shows that a modified image is rejected because secure boot checks a trusted signature. This proves the control, not an attack success.

Tools and commands used here: Firmware Mod Kit, Binwalk, dd, file, strings, hexdump, mount, GDB, and QEMU.

Countermeasures and defense: Require signed images, verify a hardware root of trust, use rollback protection, protect update storage, and reflash compromised devices from known-good media.

> **EXAM FOCUS:** Firmware modification is maintaining access, not initial access. Squashfs is a compressed filesystem; TRX is a router firmware image format. Trap: assuming password removal deletes implanted firmware.

Memory trick: "Extract, Examine, Edit, Encode, Evaluate."

3.16 Firmware analysis and reverse engineering

What it is: Static analysis inspects an image without running it. Dynamic analysis observes emulated or real execution. Reverse engineering reconstructs behavior.

Analogy: Static analysis opens and catalogs a machine; dynamic analysis runs it behind safety glass.

How it works:

1. Obtain firmware from the device or vendor and verify the MD5 value as an integrity comparison, not as collision-resistant security.
2. Run `file`, `strings`, and `hexdump` on the `.bin`; use Binwalk to identify and extract embedded data, and `dd` for precise byte ranges.
3. Create `rootfs`, mount the extracted filesystem when appropriate, and inspect `etc/passwd`, `etc/shadow`, and `etc/ssl`.
4. Search recursively for password, admin, root, `.conf`, `.pem`, and `.crt` data.
5. Emulate binaries with `qemu-mipsel`, `qemu-arm`, or the matching QEMU architecture and `-L` library prefix. Test the web interface dynamically.

Goals from the notes: passwords, API tokens, endpoints, vulnerable services, backdoor accounts, configuration files, private keys, and stored data.

Real-world use: An assessor recovers an API token from a vendor-supplied image and proves exposure without touching production cameras.

Tools and commands used here: File, MD5 tools, strings, hexdump, Binwalk, dd, mount, grep, find, QEMU, GDB, Radare2, Ghidra, and IDA Pro.

AI angle (ADDED CONTEXT): AI can suggest likely meanings for strings or decompiled functions. Prompt: "Summarize evidence in this firmware string listing and distinguish hardcoded secrets from harmless words containing 'key'."

Countermeasures and defense: Strip secrets, use per-device credentials, protect keys in secure hardware, remove debug artifacts, sign firmware, and manage third-party components.

> **EXAM FOCUS:** Binwalk extracts; QEMU emulates; strings finds printable sequences; grep searches content; find searches names. Trap: choosing QEMU for static extraction or Binwalk for full processor emulation.

Memory trick: "Get, Verify, View, Unpack, Mount, Mine, Mimic."

3.17 General IoT hacking and device-management tools

What it is: Each tool serves a different assessment surface.

Analogy: A radio stethoscope, protocol screwdriver, and router toolbox are not interchangeable just because all are used on smart devices.

How it works: CatSniffer passively monitors IoT traffic and supported CC1352-based radio protocols. KillerBee assesses Zigbee/IEEE 802.15.4. JAGulator helps identify JTAG pinouts. `wiz_exploit` is listed as an IoT exploitation project; PENIOT tests IoT device penetration; RouterSploit targets embedded devices and routers. Device management platforms onboard, monitor, permission, and remotely update fleets.

Management products in the notes: SeaCat.io, Armis Centrix, Oracle Fusion Cloud IoT, Golioth, AWS IoT Device Management, IBM Watson IoT Platform, openBalena, and Microsoft Azure IoT Central shown in the dashboard.

Real-world use: A retailer uses a management platform to inventory devices, enforce unique identities, roll out signed updates, and revoke a lost gateway.

Tools and commands used here: CatSniffer, KillerBee, JAGulator, wiz_exploit, PENIOT, RouterSploit, and the management platforms above.

Countermeasures and defense: Device management itself needs MFA, role separation, signed updates, audit logs, and secure decommissioning because it has fleet-wide power.

> **EXAM FOCUS:** Armis emphasizes asset visibility; JAGulator is JTAG; RouterSploit is embedded/router exploitation. Trap: calling a management platform a vulnerability scanner only.

Memory trick: "Cat Catches packets, KillerBee knows Zigbee, JAGulator finds JTAG."

3.18 IoT countermeasures, manufacturer guidance, and hardware practice

What it is: Defense covers accounts, network exposure, updates, physical access, and operational lifecycle.

Analogy: Good IoT security is building security plus key control plus maintenance, not one stronger lock.

The fourteen defenses in the notes:

1. Disable guest and demo accounts.
2. Lock accounts after excessive invalid logins.
3. Use strong authentication.
4. Place control networks behind firewalls and isolate them from business networks.
5. Deploy IDS/IPS.
6. Use end-to-end encryption and PKI.
7. Use VPN architecture for secure communication.
8. Deploy security as an integrated system.
9. Permit only trusted internet source IPs where appropriate.
10. Disable Telnet on port 23.

11. Disable UPnP on routers.
12. Prevent physical tampering.
13. Patch and update firmware regularly.
14. Monitor TCP 48101 because the cited infected devices spread malicious files there.

Manufacturer and OWASP solutions: Use TLS and certificate/revocation checks, strong passwords, lockout, simple secure updates, application allowlisting, and a secure boot chain. Close ports, disable UPnP, assess APIs, validate input/output, verify update source/integrity, notify users, remove unused dependencies, minimize and anonymize data, protect stored/transferred data, manage assets and decommissioning, change defaults, restrict remote access, protect BIOS/firmware with unique passwords, control boot order, and minimize USB exposure. Avoid proprietary encryption.

Hardware practices: Limit entry points, tamper protection, secure-boot monitoring, patches, interface management, restricted hardware access, protected keys and credentials, event logs, anti-malware, isolation from regular supply units, and a root of trust.

> **EXAM FOCUS:** A hardware root of trust anchors secure boot. Lockout slows online guessing but can enable lockout abuse, so combine it with monitoring and MFA. Trap: selecting UPnP disablement as a firmware-integrity control.

Memory trick: "Accounts, Access, Algorithms, Updates, Assets, Anatomy" groups identity, network, crypto, patching, management, and hardware.

3.19 OT, ICS components, and IT/OT convergence

What it is: Operational technology (OT) is hardware and software that detects or causes change in industrial operations by monitoring or controlling physical devices. An industrial control system (ICS) is the collection of systems, networks, devices, and controls that automate industrial processes.

Analogy: IT manages information; OT manages the movement and condition of the physical world.

Components:

- PLC: programmable logic controller executing local deterministic control logic.
- RTU: remote terminal unit collecting field data and issuing control at remote sites.
- SCADA: supervisory control and data acquisition across geographically distributed processes.
- DCS: distributed control system coordinating controllers within a plant/process.
- HMI: human-machine interface for display and operator commands.

- BPCS: basic process control system; SIS: safety instrumented system for safe states.
- IED: intelligent electronic device; VFD: variable-frequency drive for motor speed.
- PID: proportional-integral-derivative control used to hold a process at a setpoint.

IT/OT convergence and IIoT: Integration links OT monitoring/control with IT storage and processing such as SQL, Java, and Python to improve security, quality, efficiency, and productivity. Industry 4.0 uses these ideas for smart manufacturing. IIoT is IoT applied to industrial operations, supply chains, manufacturing, and management.

Real-world use: A plant historian sends carefully brokered production data to IT analytics without allowing the analytics server to directly write to a safety PLC.

Tools and commands used here: Malcolm, Wireshark, Nmap OT scripts, Nessus, Skybox, and Microsoft Defender for IoT support later assessments.

AI angle (ADDED CONTEXT): Predictive maintenance models can detect failing motors. Poisoned sensor training data could hide failure. Prompt: "Identify data-quality checks for this maintenance dataset and do not recommend changes to live control setpoints."

Countermeasures and defense: Keep safety functions independent, broker IT/OT data through controlled zones, and design for availability and safe failure.

> **EXAM FOCUS:** HMI is the operator window; PLC is local logic; SCADA supervises wide-area operations; DCS distributes plant control. Trap: treating SIS as an ordinary business availability system instead of a safety system.

Memory trick: "PLC Performs, RTU Reaches, SCADA Supervises, DCS Distributes, HMI Helps Humans."

3.20 Purdue model and OT protocol placement

What it is: The Purdue Enterprise Reference Architecture (PERA) inspired a model of ICS zones and dependencies. The enterprise zone and manufacturing zone are separated by an industrial DMZ (IDMZ).

Analogy: A factory has public offices, a controlled reception, operations rooms, supervisor desks, machine controllers, and the machines themselves.

Levels:

1. Level 5: enterprise network.
2. Level 4: business logistics systems.
3. Level 3.5: IDMZ, the brokered boundary between IT and OT.
4. Level 3: operations systems/site operations.
5. Level 2: control systems/area supervisory control.

6. Level 1: basic control/intelligent devices.
7. Level 0: physical process, sensors, and actuators.

Protocol mapping from the notes:

- Levels 4/5: DCOM, FTP/SFTP, GE-SRTP, IPv4/IPv6, OPC UA, TCP/IP, Wi-Fi, SMTP, HTTP/HTTPS.
- Level 3: CC-Link, GE-SRTP, HSCP, ICCP (IEC 60870-6), IEC 61850, IEC 60870-5-104, ISA/IEC 62443, Modbus, NTP, Profinet, SuiteLink, Tase.2, TCP/IP, ControlNet, Profibus PA/DP.
- Level 2: 6LoWPAN, CC-Link, DNP3, DNS/DNSSEC, FTE, HART-IP, IEC 60870-5-101/104, IPv4/IPv6, ISA/IEC 62443, OPC, NTP, SOAP, TCP/IP, DeviceNet, AS-Interface.
- Levels 0/1: BACnet, EtherCAT, CANopen, Crimson v3, GE-SRTP, Zigbee, ISA/IEC 62443, ISA SP100, MELSEC-Q, Modbus, Niagara Fox, Omron FINS, PCWorx, Profibus, Profinet, Sercos II, S7 Communications, WiMAX, and FOUNDATION Fieldbus.

> **ADDED CONTEXT:** ISA/IEC 62443 is a security standard series, not a wire protocol. Protocols can appear at multiple levels depending on architecture. Learn the slide's representative placement, not an absolute rule that a protocol can exist at one level.

Real-world use: A design team places a patch broker and historian replica in the IDMZ rather than connecting the enterprise directly to Level 1 controllers.

Tools and commands used here: Firewalls, data diodes where suitable, protocol-aware monitoring, Malcolm, and securely configured jump hosts.

Countermeasures and defense: Define zones by function and risk, permit only required conduit traffic, and block direct enterprise-to-controller access.

> **EXAM FOCUS:** Level 0 is physical process, not basic controller; Level 3.5 is IDMZ. Trap: placing business email at Level 2 or a PLC at Level 4.

Memory trick: "Five Enterprise, Four Business, Three Operations, Two Supervise, One Control, Zero Physical."

3.21 OT challenges and vulnerabilities

What it is: OT risk comes from old technology and from connections added faster than governance and protection.

Analogy: A historic machine newly connected to the internet has modern exposure but may still have decades-old locks.

The 22 challenges in notes order: lack of visibility; haphazard modernization; vulnerable communication protocols; plain-text passwords; network complexity; legacy technology; lack of antivirus; lack of skilled security professionals; rapid change; outdated systems; insecure connections; rogue devices; IT convergence; organizational challenges; unique production networks/proprietary software; remote management protocols; insufficient segmentation; physical security; vendor dependencies; resource constraints; lack of encryption; and data-integrity issues.

The ten vulnerability lessons: Publicly accessible OT permits brute force or disruption. Insecure jump boxes expose remote access. Missing updates leave known flaws. Default/weak credentials open management interfaces. Poor firewall rules spread threats. Compromised IT can reach OT placed in the corporate network, and insecure OT can also expose IT. Flat networks allow one compromise to expose all. Wireless without strong encryption/authentication permits sniffing and bypass. Unrestricted outbound OT internet access enables malware and C2.

Real-world use: A risk review finds a vendor jump host with shared credentials and direct controller access. The best fix is layered managed remote access, not just changing the shared password once.

Tools and commands used here: Asset inventory, passive monitoring, firewall/ACL review, vulnerability assessment, and identity management.

Countermeasures and defense: Compensating controls matter when patching is not immediately possible: isolation, allowlisting, virtual patching through IPS, managed access, and tested backups.

> **EXAM FOCUS:** A flat network has insufficient segmentation. Outbound C2 needs egress restriction, not only inbound firewalling. Trap: choosing immediate patching without testing when the question stresses a fragile continuous process.

Memory trick: "Old, Open, Overconnected, Unowned, Unpatched, Unobserved."

3.22 MITRE ATT&CK for ICS

What it is: MITRE ATT&CK for ICS is a knowledge base of adversary tactics, the operational goals, and techniques, the ways those goals are achieved.

Analogy: Tactics are chapters in an intruder's plan; techniques are the specific moves used in each chapter.

Tactics and examples from the notes:

1. Initial access: drive-by compromise, public application or remote-service exploit.
2. Execution: change operating mode, CLI, or execution through APIs.
3. Persistence: program modification, module firmware, APIs, project-file infection.
4. Privilege escalation: software exploitation and hooking.

5. Evasion: remove indicators, rootkits, or change operator mode.
6. Discovery: network enumeration, sniffing, and remote-system identification.
7. Lateral movement: default credentials, program download, and remote services.
8. Collection: automated collection, information repositories, and I/O images.
9. Command and control: common ports, connection proxy, standard application protocol.
10. Inhibit response function: firmware update mode or block command/report messages.
11. Impair process control: I/O brute force, parameter alteration, module firmware.
12. Impact: property damage, loss of availability, and denial of control.

Real-world use: A SOC maps a blocked reporting message to inhibit response function and checks whether safety alarms still reach operators.

Tools and commands used here: ATT&CK Navigator, logs, network detection, asset inventory, and incident-response procedures.

AI angle (ADDED CONTEXT): AI can propose ATT&CK mappings from observed events. Prompt: "Map these verified ICS events to tactics, cite the event text, and distinguish inhibit response function from impair process control."

Countermeasures and defense: Use behavior-based detections across several tactics, independent safety protections, immutable logs, and rehearsed manual operations.

> **EXAM FOCUS:** Blocking alarms inhibits response; changing process parameters impairs control; damage or denial of control is impact. Trap: treating discovery as initial access when the attacker is already inside.

Memory trick: "Inside Everyone Plants Persistent Evidence; Discovery Leads Collection; Control Interrupts Processes Immediately."

3.23 OT threats, HMI attacks, and side channels

What it is: The OT threat list is maintenance/administrative threats, data leakage, protocol abuse, destruction of ICS resources, reconnaissance, DoS, HMI attacks, enterprise-system/tool exploitation, spear phishing, malware, unpatched vulnerabilities, side channels, buffer overflows, and RF remote-controller abuse.

Analogy: The HMI is the plant's control-room dashboard. Compromising it can hide the real road and send wrong driving instructions.

How HMI compromise works: Attackers target the central visualization/control hub to damage SCADA devices or steal architecture information. Memory corruption includes out-of-bounds read/write and heap/stack buffer overflow. Credential

weaknesses include hardcoded and cleartext passwords. Cleartext data, insecure defaults, weak authentication/authorization, unsafe ActiveX controls, missing encryption, and code injection enable further compromise.

Side-channel methods: Timing analysis compares authentication time to infer correct password characters. Power analysis uses an oscilloscope/measuring probe and analysis software to compare power consumption across clock cycles and infer character values.

Real-world use: A vendor removes hardcoded HMI credentials and tests patched runtime software against malformed project files before plant deployment.

Tools and commands used here: Debuggers, fuzzers, oscilloscopes, signal analyzers, and HMI/SCADA vulnerability scanners.

Countermeasures and defense: Patch HMI software, remove ActiveX dependence, use memory-safe development and input validation, unique credentials, MFA at access boundaries, constant-time authentication, and power/timing leakage resistance.

> **EXAM FOCUS:** Buffer overflow is memory corruption; cleartext credential storage is credential management. Trap: choosing network sniffing when the evidence is oscilloscope power traces.

Memory trick: "HMI Holds Manipulation and Information"; "Time Tells, Power Paints."

3.24 PLC rootkits, Evil PLC, RF controllers, and supply chains

What it is: A PLC rootkit hides in controller execution and manipulates I/O. An Evil PLC attack weaponizes a PLC against engineering workstations. Industrial RF attacks abuse remote-control transmitters. Supply-chain attacks arrive through trusted hardware, software, services, or relationships.

Analogy: A rootkit changes the machine's reflexes. Evil PLC turns the machine into a trap for its mechanic. Re-pairing replaces the machine's recognized remote.

PLC rootkit sequence:

1. Discover an exposed PLC with Shodan or obtain access through default credentials.
2. Gain root/control-flow access and map the physical process plus I/O-pin/logic links.
3. Modify I/O initialization, debug registers, and intercept read/write operations.
4. Manipulate process integrity and availability while hiding the real state.

Other methods: Evil PLC changes PLC configuration/logic through download procedures; connected engineering workstations later initiate upload procedures and can be attacked, disrupting production. RF replay records and resends commands. RF

command injection reverse-engineers and alters or creates packets. Malicious re-pairing pairs a hostile transmitter. Malicious reprogramming implants controller firmware for persistent remote access.

Supply-chain types: third-party software compromise, hardware manipulation, service-provider breach, malicious-component injection, and exploitation of trusted relationships.

Real-world use: A plant requires engineering workstations to treat PLC project files as untrusted and verifies controller logic against signed known-good versions.

Tools and commands used here: Shodan, SDR equipment, debuggers, firmware analysis, Metasploit, and modbus-cli.

Countermeasures and defense: Restrict programming modes, authenticate project downloads, separate engineering workstations, validate logic, protect pairing, sign firmware, assess suppliers, and verify received hardware/software.

> **EXAM FOCUS:** Evil PLC targets the workstation through trusted PLC interactions; a PLC rootkit primarily hides and manipulates controller I/O. Trap: choosing replay when the attacker creates new RF packets rather than resending captured ones.

Memory trick: RF methods are "Repeat, Rewrite, Re-pair, Reprogram."

3.25 OT malware: Fuxnet and related families

What it is: OT malware can disrupt industrial operations, destroy data, block sensor gateways, or attempt to damage sensors.

Analogy: Ordinary ransomware locks the office records; OT destructive malware can also interfere with the machinery outside the office.

How it works: Fuxnet is described as destructive ICS malware that modifies crucial data, prevents access to sensor gateways, and attempts physical sensor damage. The notes show deployment-log fragments, a list of sensor-router IPs, `xargs` parallel execution, and JSON inventory containing IPs, a port, and collector/control-room objects. Read these as evidence of targeting and deployment, not as a recipe.

Other listed malware: Kapeka, Abyss Locker, AvosLocker, COSMICENERGY, INDUSTROYER.V2, and Pipedream.

> **ADDED CONTEXT:** Kapeka is a backdoor; Abyss Locker and AvosLocker are ransomware; COSMICENERGY was designed for electric-grid disruption; INDUSTROYER.V2 targets industrial communication, notably IEC-104; Pipedream is modular ICS malware.

Real-world use: A utility prioritizes gateway reachability, immutable historian backups, and manual fallback because malware may affect both data and sensor access.

Tools and commands used here: EDR/anti-malware, passive ICS monitoring, backup and recovery systems, and log analysis.

Countermeasures and defense: Segment gateways, restrict engineering changes, maintain offline backups, rehearse manual operation, and monitor unauthorized writes or firmware activity.

> **EXAM FOCUS:** Fuxnet is destructive and sensor/gateway focused; Pipedream is a modular ICS toolkit. Trap: assuming all listed OT malware is financially motivated ransomware.

Memory trick: "Fuxnet Fouls Field sensors and gateways."

3.26 OT information gathering with Shodan and CIRT.net

What it is: Public search can reveal exposed SCADA and vendor products. Default password databases reveal vendor-shipped credentials to check and replace.

Analogy: Public banners reveal the model of a door; CIRT.net tells you which key the manufacturer originally supplied.

How it works:

1. Shodan `port:502` finds Modbus services; `"Schneider Electric"` searches a PLC vendor; `SCADA country:"US"` combines product and geography.
2. Results may include IPs, organizations, ports, and service banners. The example also contains FTP response text, proving that Shodan results are not all Modbus.
3. CIRT.net lists default passwords for routers, switches, ICS, and other devices, including Siemens SIMATIC WinCC SCADA examples.
4. Kamerka-GUI maps exposed devices, SearchDiggity supports search-based discovery, Zeek analyzes network traffic, Criminal IP and ZoomEye provide exposure intelligence.

Real-world use: An owner searches only its authorized public range and verifies that default WinCC credentials no longer work through a controlled configuration review.

Tools and commands used here: Shodan, CIRT.net, Kamerka-GUI, SearchDiggity, Zeek, Criminal IP, and ZoomEye.

Countermeasures and defense: Remove direct exposure, change defaults at install, restrict management access, and monitor for unexpected public services.

> **EXAM FOCUS:** `port:502` is a service filter, not proof of a specific PLC vendor. CIRT.net is password intelligence; Shodan is internet exposure intelligence.

Memory trick: "Shodan Shows Systems; CIRT Shows Secrets shipped by default."

3.27 OT protocol discovery with Nmap, Wireshark, and Malcolm

What it is: Nmap sends probes and runs scripts. Wireshark decodes packets. Malcolm combines network-traffic analysis, Zeek logs, OpenSearch dashboards, and Arkime.

Analogy: Nmap knocks cautiously, Wireshark reads one conversation, and Malcolm builds an operations dashboard from many conversations.

How it works: Slow TCP discovery checks HMI and PLC ports; UDP scripts query EtherNet/IP and BACnet. Nmap scripts identify S7, Modbus, Niagara Fox, ProConOS, Omron, and PCWorx. Wireshark's Modbus example shows transaction ID, protocol ID, length, unit ID, function code Read Input Registers, reference number, and word count. Malcolm counts protocols such as Modbus, DNP3, EtherNet/IP, BACnet, and S7comm.

> **REMEMBER:** Modbus/TCP has no built-in encryption or authentication in its classic form. Reading or writing requires only network reachability in many deployments.

Real-world use: Before any active scan, a team reviews a SPAN capture in Malcolm to discover controllers and builds a carefully scoped maintenance-window scan.

Tools and commands used here: Nmap NSE scripts, Wireshark, Malcolm, Zeek, OpenSearch, and Arkime.

AI angle (ADDED CONTEXT): AI can summarize protocol baselines but must not decide that active probing is safe. Prompt: "From this approved Malcolm protocol inventory, list unexpected cross-zone conversations and propose passive validation only."

Countermeasures and defense: Use passive asset discovery, firewall industrial ports, allow only required masters, monitor function codes, and test active scripts in a lab before production use.

> **EXAM FOCUS:** `enip-info` uses UDP 44818; `bacnet-info` uses UDP 47808; `modbus-discover` uses TCP 502; `s7-info` uses TCP 102. Trap: running fast parallel discovery against fragile controllers when the safest first step is passive capture.

Memory trick: "Nmap Names, Wireshark Reads, Malcolm Maps."

3.28 Nessus, Fuzzowski, and Skybox

What it is: Nessus maps systems to known vulnerabilities. Fuzzowski sends malformed or varied protocol inputs to discover errors. Skybox correlates vulnerabilities, network paths, exposure, and risk across IT/OT.

Analogy: Nessus checks a recall catalog, fuzzing tests unexpected handling, and Skybox asks which defective part is both reachable and important.

Nessus policy steps from the notes:

1. Log in, open Policies, create a Basic Network Scan policy.
2. Set Discovery port range 0-1000 for the demonstration.
3. Check that SCADA plugins are enabled/present, or results cover non-SCADA ports only.
4. Save, create a scan from the user-defined policy, enter the target, and launch.
5. Review grouped findings such as ICCP/COTP, Modbus/TCP, and EtherNet/IP identity.

Fuzzing: BACnet uses UDP 47808, Modbus TCP 502, and IPP TCP 631. Fuzzowski selects a protocol fuzzer with `-f`, transport with `-p`, requests with `-r`, receive timeout with `-rt`, monitors with `-m`, and restart behavior with `--restart`.

Real-world use: A team fuzzes a vendor protocol implementation in an isolated lab, then uses Skybox to prioritize the corresponding production exposure without fuzzing the live controller.

Tools and commands used here: Nessus, Fuzzowski, Skybox, SmartRF Packet Sniffer, and Microsoft Defender for IoT.

Countermeasures and defense: Require maintenance approval, backups, vendor support, and rollback. Prefer passive or credentialed assessment when active probing threatens availability.

> **EXAM FOCUS:** Nessus is vulnerability assessment; Fuzzowski is protocol robustness testing; Skybox is exposure/path/risk prioritization. Trap: treating a scan policy with no SCADA plugins as proof that no SCADA vulnerabilities exist.

Memory trick: "Nessus Knows known flaws; Fuzzowski Feeds funny frames; Skybox Scores paths."

3.29 ICS hardware analysis, Metasploit, modbus-cli, and DNP3

What it is: Hardware analysis examines chips and their integrated software. Modbus tools discover unit IDs and read/write registers or coils. DNP3 exposure can permit unauthorized remote access when combined with weak configuration.

Analogy: A register is a numbered storage box for a multi-bit value; a coil is a one-bit switch. A unit ID identifies which slave device receives the request.

How it works:

1. Static/dynamic chip analysis reveals function arguments, missing I/O validation, certificates, key generation, and encryption information.
2. Metasploit's `modbus_findunitid` probes unit IDs; correct responses identify active stations. `modbusclient` selects address, count, host, and unit to read registers.

3. Install modbus-cli with RubyGems. Read `%MW100` or `400101` for registers and `%M100` or `101` for coils. Datatypes include word, signed integer, float, dword, and Boolean coil. Save output with `--output`.
4. Shodan searches such as `TM221ME16R` identify specific Schneider PLC models.
5. For DNP3, discover open services and weak/default access, but validate only with explicit approval because writes can change a process.

Hardware tools: Software: GDB, OpenOCD, Binwalk, Fritzing, Radare2, Ghidra, IDA Pro. Hardware: signal analyzer, multimeter, memory programmer, microcontroller, oscilloscope, soldering tools, microscope, JTAG interface, screwdrivers, tweezers, and converters.

Real-world use: In a testbed, an assessor reads five holding values and demonstrates that network access alone exposes process data, then recommends firewall and monitoring controls rather than changing live values.

Tools and commands used here: Metasploit auxiliary modules, modbus-cli, Shodan, Nmap, Binwalk, GDB, OpenOCD, and hardware instruments.

Countermeasures and defense: Restrict port 502 and DNP3 port 20000 to authorized masters, authenticate remote access externally, log write function codes, segment, and require programming-mode authorization.

> **EXAM FOCUS:** `%MW` denotes a 16-bit memory word in the shown Schneider syntax; `%M` denotes a coil. `400101` is a Modbus reference form. Trap: confusing register values with one-bit coils or using UDP for classic Modbus TCP.

Memory trick: "Find the Unit, Set the Host, Read the Register, Mind the Coil."

3.30 OT defenses, Purdue controls, zero trust, and organizations

What it is: OT defense is layered risk management that protects operations without unsafe testing or uncontrolled change.

Analogy: A plant needs gates, visitor logs, trained operators, emergency stops, and rehearsals, not just a taller fence.

Core defenses from the notes: Purpose-built passive sensors; supported technology and tested patches; secure configuration/coding; an asset register; strong changed passwords; VPN-backed layered remote access; perimeter filtering; anti-malware; unused-service hardening; manufacturer patches; IDS and flow measurement; tested third-party web servers; cryptographically signed updates; and periodic audits.

Vulnerability-to-solution pairs: Public exposure needs enterprise firewalls, managed remote access, MFA, and incident response. Remote connections need strong identity, RBAC, and patching. Updates need sandbox testing and hardening. IT/OT connections need segregation, an IDMZ, business-based rules, and secure

gateways. Flat OT needs critical/noncritical separation, zones, defense in depth, and zero trust. Wireless needs standard strong encryption. Outbound access needs risk assessment, monitoring, segregation, and a separate update repository. Firewalls need secure configuration and ACLs. IT and OT should use separate username conventions.

Purdue security controls: Levels 4/5 use firewalls, IPS, anti-bot, URL filtering, SSL inspection, antivirus, and DLP. Level 3.5 uses anti-DoS, IPS, anti-bot, application control, and application-layer firewalling. Level 3 adds sandboxing, traffic encryption, monitoring, and USB/port protection. Levels 1/2 use IPS, firewalls, IPsec, security gateways, and authorized RTU/PLC commands. Level 0 adds point-to-point controls, MAC authentication, and additional gateways at Levels 0/1.

Zero-trust sequence: Define the network, map traffic, architect the network, develop the zero-trust policy, then monitor and maintain. Verify users, applications, and devices continuously and grant only required access.

Organizations: OTCC shares best practices among industry and government. OT-ISAC supports operational-technology threat information sharing. NERC develops and enforces electric reliability standards in its jurisdiction.

> **EXAM FOCUS:** Zero trust is not one appliance. It is identity, least privilege, segmentation, and continuous validation. Trap: allowing a vendor permanent broad access because the vendor is trusted.

Memory trick: Zero trust: "Define, Draw, Design, Decide, Detect."

4. Tool Encyclopedia

4.1 Exposure search and information gathering

Tool	Simple purpose	Type	Phase	Exam keyword
Shodan	Find exposed service banners	Web search	Recon	Internet-connected PLC
Censys	Search internet assets	Web search	Recon	Alternative to Shodan
FOFA	Search cyberspace assets	Web search	Recon	Exposure mapping
ZoomEye	Search exposed devices	Web search	Recon	CamOver API option
Criminal IP	Assess IP exposure	Web intelligence	Recon	IP risk intelligence
MultiPing	Monitor many hosts	Network utility	Discovery	Multiple-host latency
FCC ID Search	Identify radio approval data	Public database	Recon	Find RF frequency
CIRT.net	Find factory passwords	Password database	Recon	Default WinCC password
Kamerka-GUI	Map exposed devices	OSINT GUI	Recon	Geolocation and ICS
SearchDiggity	Query exposed resources	OSINT suite	Recon	Search-engine dorks
Zeek	Produce network logs	NSM engine	Monitoring	Connection and protocol logs
iRidium	Visualize control systems	HMI/ICS software	Target example	HVAC control panel

4.2 Discovery, scanning, analysis, and prioritization

Tool	Simple purpose	Type	Phase	Exam keyword
Nmap	Discover ports and services	Active scanner	Scanning	NSE `s7-info`
Wireshark	Decode captured packets	Protocol analyzer	Analysis	Modbus function code

Tool	Simple purpose	Type	Phase	Exam keyword
Malcolm	Analyze ICS network traffic	NSM platform	Discovery	OpenSearch and Arkime
Arkime	Index and search packets	Packet interface	Analysis	Malcolm packet view
OpenSearch	Search and visualize logs	Dashboard	Analysis	Malcolm dashboards
Nessus	Find known vulnerabilities	Vulnerability scanner	Assessment	SCADA plugins
IoTSeeker	Check default IoT passwords	Perl scanner	Assessment	Camera default passwd
Genzai	Identify IoT flaws	IoT toolkit	Assessment	Paths and versions
beSTORM	Fuzz and test software	Security tester	Assessment	Protocol robustness
IoTSploit	Test IoT targets	Exploitation suite	Assessment	IoT framework
IoTVAS	Assess IoT firmware	Analysis platform	Assessment	Firmware risk
Firmalyzer	Analyze firmware	Firmware platform	Assessment	Firmware components
Skybox	Prioritize exposed risk	Risk/path analysis	Assessment	Combined IT/OT paths
Defender for IoT	Monitor IoT/OT assets	Security platform	Defense	Passive asset visibility
SmartRF Sniffer	Capture TI radio packets	RF analyzer	Analysis	TI wireless protocols

4.3 Radio, hardware, and exploitation tools

Tool	Simple purpose	Type	Phase	Exam keyword
RFCrack	Test sub-GHz signals	Python SDR tool	Attack test	Rolling code `-r`
YardStick One	Send/receive sub-GHz RF	USB radio	Attack test	RFCrack hardware
HackRF One	Capture/transmit wideband RF	Half-duplex SDR	Attack test	1 MHz to 6 GHz
RTL-SDR	Receive radio spectrum	Low-cost SDR	Recon	Find nearby frequency

Tool	Simple purpose	Type	Phase	Exam keyword
Open Sniffer	Capture/send 802.15.4	Zigbee sniffer	Attack test	Replay Zigbee frames
CatSniffer	Passively monitor IoT RF	Multi-protocol tool	Recon	CC1352 traffic
KillerBee	Assess Zigbee networks	Python/RF framework	Attack test	802.15.4 security
Suphacap	Capture wireless traffic	Sniffer	Analysis	Listed IoT sniffer
BUS Auditor	Find UART/I2C/ JTAG pins	Hardware scanner	Recon	RX TX and baud
DIVA	Practice hardware security	Training target	Lab	Insecure board app
JAGulator	Identify JTAG pinout	Hardware tool	Recon	JTAG pin discovery
CamOver	Test camera disclosure	Exploit tool	Gaining access	`-a` camera IP
RouterSploit	Test embedded devices	Exploit framework	Gaining access	Router exploits
PENIOT	Test IoT device security	Pentest framework	Assessment	IoT penetration
wiz_exploit	Test listed IoT issues	Exploit project	Gaining access	Notes tool list
Metasploit	Run security modules	Exploit framework	Assessment	`modbus_finduniti d`
modbus-cli	Read/write Modbus data	Ruby CLI	Assessment	`%MW100` register
Fuzzowski	Fuzz network protocols	Python fuzzer	Assessment	`-f` and `--restart`

4.4 Firmware and reverse-engineering tools

Tool	Simple purpose	Type	Phase	Exam keyword
Firmware Mod Kit	Extract/rebuild firmware	Firmware toolkit	Maintain access	`extract-firmware.sh`
Binwalk	Identify/extract images	Firmware analyzer	Analysis	Embedded Squashfs
Redress	Inspect Go binaries	Reverse tool	Malware analysis	Go source functions

Tool	Simple purpose	Type	Phase	Exam keyword
GDB	Debug program execution	Debugger	Analysis	Execution flow
OpenOCD	Drive debug hardware	Debug server	Analysis	JTAG debugging
Fritzing	Document circuit layouts	Electronics design	Analysis	PCB prototyping
Radare2	Reverse binaries	RE framework	Analysis	Disassembly
Ghidra	Decompile and analyze	RE suite	Analysis	NSA reverse suite
IDA Pro	Disassemble/ decompile	Commercial RE	Analysis	Static binary review
QEMU	Emulate processor/system	Emulator	Dynamic analysis	`qemu-mipsel`
Signal analyzer	Measure electronic signals	Instrument	Hardware analysis	Chip behavior
Oscilloscope	Display voltage over time	Instrument	Side channel	Power trace
Multimeter	Measure electrical values	Instrument	Hardware recon	Ground pin and voltage

4.5 Device management and security products

Tool	Simple purpose	Type	Main use	Exam keyword
SeaCat.io	Manage connected devices	IoT platform	Operations	Secure device access
Armis Centrix	Discover and manage assets	Asset security	Visibility	Agentless IoT/OT assets
Oracle Fusion IoT	Manage IoT fleet/ data	Cloud platform	Management	Oracle IoT
Golioth	Manage embedded fleets	Cloud platform	Management	Firmware and devices
AWS IoT Device Mgmt	Onboard and update devices	Cloud platform	Management	AWS fleet jobs
IBM Watson IoT	Connect and analyze devices	Cloud platform	Management	IBM IoT Platform
openBalena	Deploy/manage device fleets	Open platform	Management	Balena self-hosted

Tool	Simple purpose	Type	Main use	Exam keyword
Azure IoT Central	Operate IoT applications	Cloud platform	Management	Azure dashboard

5. Command Sheet

All examples are for systems you own or are explicitly authorized to assess. Sample outputs are simplified teaching examples. IPs are private/documentation values.

5.1 Shodan filters and HTTP discovery with Nmap

Purpose: Locate candidate exposed devices, then identify selected web ports in an authorized scope.

```
webcamxp country:"US"
webcamxp city:"paris"
webcamxp geo:"-50.81,201.80"
port:502
"Schneider Electric"
SCADA country:"US"
TM221ME16R
```

```
nmap -p 80,81,8080,8081 192.168.10.0/24
```

Part	Meaning	When used	Example
`-p`	Select ports	Known web alternatives	`-p 80,8080`
target	IP, host, or range	Authorized scope only	`192.168.10.20`
`country:`	Shodan country filter	Geographic exposure	`country:"US"`
`port:`	Shodan service port	Find Modbus candidates	`port:502`

```
PORT     STATE  SERVICE
80/tcp   open   http
81/tcp   closed unknown
8080/tcp open   http-proxy
8081/tcp closed blackice-icecap
```

Open means a service accepted the probe; closed means reachable but no listener on that port. The service label is a guess based on port/probe, not proof of device type. Mistakes include using uppercase `-P` for ports and assuming 8080 proves a camera. The exam asks you to distinguish Nmap active scanning from Shodan indexed results.

5.2 ifconfig, Airmon-ng, Airodump-ng, and Wireshark

Purpose: Identify a wireless adapter, enable monitor mode, discover networks, and capture one channel.

```
ifconfig
airmon-ng start wlan0
airodump-ng wlan0mon
airmon-ng start wlan0mon 11
```

Command/part	Meaning	When used	Example
`ifconfig`	Show interface configuration	Find adapter name	`wlan0`
`start`	Enable monitor mode	Before capture	`airmon-ng start wlan0`
`wlan0mon`	Monitor interface	Airodump/Wireshark source	Capture target
`11`	Lock channel 11	Match target network	Last argument above

[UNCLEAR on page 11] The transcription shows `airodump-ng start wlan0mon`; the best reading is `airodump-ng wlan0mon`, because Airodump-ng does not need Airmon's `start` subcommand. It also shows `nmap -P`; standard Nmap port selection is `-p`.

```
CH 11  BSSID  AA:BB:CC:DD:EE:FF  PWR -42  ESSID LabSensors
MQTT Ping Request
MQTT Publish Message
TCP dst port 1883
```

Read BSSID as the access-point radio MAC and ESSID as network name. MQTT keepalive and publish packets at 1883 suggest plaintext MQTT. Common mistakes are capturing the wrong channel, choosing the non-monitor interface, and treating a protocol name as proof of readable payload. The exam tests `airmon-ng` versus `airodump-ng`.

5.3 IoTSeeker

Purpose: Check listed devices for factory credentials.

```
perl IotScanner.pl 192.168.10.31,192.168.10.43,192.168.10.53
```

Part	Meaning	When used	Example
`perl`	Run Perl interpreter	Script has no wrapper	First token
`IotScanner.pl`	IoTSeeker scanner	Authorized device list	Script name
comma list	Targets to test	Known addresses	No spaces shown

```
device 192.168.10.31 is of type Arecont has changed passwd
device 192.168.10.43 is of type W-Box still has default passwd
```

The first host does not retain its factory password; the second is high priority. Vendor names identify detected device types. Mistakes include scanning a whole internet list and treating a changed password as proof of strength. Exam keyword: "default, factory-set credentials" points to IoTSeeker.

5.4 RFCrack

Purpose: Test live replay, rolling-code behavior, RSSI filtering, and jamming on owned sub-GHz equipment.

```
python RFCrack.py -i
python RFCrack.py -r -M MOD_2FSK -F 314350000
python RFCrack.py -r -M MOD_2FSK -F 314350000 -U -100 -L -10
python RFCrack.py -j -F 314000000
```

Flag	Meaning	When used	Example
`-i`	Live replay mode	Immediate capture/ replay	`RFCrack.py -i`
`-r`	Rolling-code mode	Test jam/capture sequence	With `-M` and `-F`
`-M`	Modulation	Match target signal	`MOD_2FSK`
`-F`	Frequency in Hz	Match transmitter	`314350000`
`-U`	Upper RSSI bound	Filter captured signals	`-100`
`-L`	Lower RSSI bound	Filter captured signals	`-10`
`-j`	Jamming mode	Controlled RF lab only	With frequency

```
Welcome to RFCrack - A Software Defined Radio Attack Tool
Release: 1.4
Hardware Needed: 1 YardStick or 2 for RollingCode
```

The banner identifies the tool/version and warns that rolling-code testing can need two radios. Do not confuse 314350000 Hz with 314.35 MHz notation. The notes' exact RSSI bound semantics are tool-specific; remember the values as shown. Common errors are choosing `-i` for rolling code or omitting modulation. The exam commonly asks which flag performs jamming or rolling-code mode.

5.5 HackRF transfer and device information

Purpose: Capture raw quadrature radio samples, replay them, and identify hardware.

```
hackrf_transfer -r capture.raw -f 433900000 -l 20 -g 20
hackrf_transfer -t capture.raw -f 433900000 -x 40
hackrf_info
```

Flag	Meaning	When used	Example
`-r`	Receive to file	Capture signal	`-r capture.raw`
`-t`	Transmit from file	Replay owned signal	`-t capture.raw`
`-f`	Center frequency Hz	Tune radio	`433900000`
`-l`	RX LNA gain dB	Adjust RF input gain	`20`
`-g`	RX VGA gain dB	Adjust baseband gain	`20`
`-x`	TX VGA gain dB	Adjust transmit gain	`40`
`-s`	Sample rate Hz	Set capture bandwidth	`10000000`

```
call hackrf_set_freq(433900000 Hz/433.900 MHz)
19.9 MiB / 1.000 sec = 19.9 MiB/second
Board ID Number: 2 (HackRF One)
Firmware Version: 2015.07.2
```

Frequency confirms tuning, throughput confirms streaming, and info identifies board and firmware. The source capture's filenames are blurry; `capture.raw` here is a clear teaching substitute. LNA gain must fit device increments. Never transmit outside authorized frequencies/labs. Exam focus: `-r` receives, `-t` transmits, `-f` tunes.

5.6 BUS Auditor UART and I2C scans

Purpose: Identify hardware pins and communication settings.

```
run busauditor.generic.uartscan -v 3.3 -p /dev/ttyACM0
run busauditor.generic.i2cscan -v 3.3 -p /dev/ttyACM0 -s 0 -e 10
```

Part	Meaning	When used	Example
`uartscan`	Search UART combinations	Find console pins	Generic module
`i2cscan`	Search I2C combinations	Find device/address	Generic module
`-v`	Target voltage	Match board logic	`3.3`
`-p`	Serial device	Connect BUS Auditor	`/dev/ttyACM0`
`-s`	Start pin	Limit scan range	`0`
`-e`	End pin	Limit scan range	`10`

```
UART Scan Result: RX pin 1, TX pin 0, BaudRate 9600
I2C Scan Result: Device Address 0x48, SCL 1, SDA 0
```

RX/TX are receive/transmit, baud is serial speed, 0x48 is a hexadecimal device address, SCL is clock, and SDA is data. Wrong voltage can damage equipment. The exam asks which interface uses SCL/SDA and which scan finds baud rate.

5.7 CamOver

Purpose: Test susceptible network cameras for administrative password disclosure.

```
camover
camover -a 192.168.10.50
camover -t --shodan SHODAN_API_KEY
camover -i addresses.txt -o results.txt
```

Flag	Meaning	When used	Example
`-h`	Show help	Review options	`camover -h`
`-t`	Use threads	Faster approved tests	With input/API
`-o`	Output result file	Preserve evidence	`-o results.txt`
`-i`	Input address file	Multiple owned targets	`-i addresses.txt`
`-a`	Single address	One camera	`-a 192.168.10.50`
`--shodan`	Shodan API key	Internet research scope	Do not expose key
`--zoomeye`	ZoomEye API key	Alternate search	Do not expose key
`-p`	ZoomEye pages	Limit result pages	`-p 1`

```
CamOver is a camera exploitation tool
usage: camover [-h] [-t] [-o OUTPUT] [-i INPUT] [-a ADDRESS]
```

Help confirms that `-a` is one address and `-t` is threading. Real output depends on camera vulnerability; no response is not proof of security. Common mistakes are using `-t` as target and placing an API key in shared notes. The exam tests tool selection and `-a` for one IP.

5.8 Firmware Mod Kit extraction

Purpose: Deconstruct a firmware image for static review and controlled rebuilding.

```
./extract-firmware.sh dd-wrt.v24_micro_generic.bin
```

Part	Meaning	When used	Example
`./`	Run in current directory	Script is not on PATH	Before script

Part	Meaning	When used	Example
script	FMK extraction routine	Known firmware image	`extract-firmware.sh`
argument	Input image	Vendor/owned firmware	`.bin` file

```
0x0     TRX firmware header, little endian
0x1C    gzip compressed data
0x9A8   LZMA compressed data
70728   Squashfs filesystem, little endian
Extracting squashfs file system at offset 670720
```

Offsets show where components begin; compression identifies storage encoding; Squashfs contains the embedded filesystem. An MD5 checksum identifies the analyzed image but is not a trusted signature. Mistakes include modifying the only original and assuming successful extraction means successful repacking/boot. Exam focus: FMK deconstructs and reconstructs firmware.

5.9 Static firmware commands and QEMU emulation

Purpose: Inspect, extract, mount, search, and emulate firmware.

```
file firmware.bin
md5sum firmware.bin
strings firmware.bin
hexdump firmware.bin
binwalk firmware.bin
dd if=firmware.bin of=rootfs.img bs=1 skip=670720
mkdir rootfs
sudo mount -t ext2 rootfs.img rootfs
grep -rnw rootfs -e password
find rootfs -name "*.conf"
qemu-mipsel -L rootfs ./binary
qemu-arm -L rootfs ./binary
```

Command/flag	Meaning	When used	Example
`file`	Identify file type	First triage	`.bin` input
`md5sum`	Calculate MD5 digest	Match known image	Not a signature
`strings`	Show printable text	Find clues/secrets	Firmware image
`hexdump`	Show raw bytes	Inspect header	Firmware image
`binwalk`	Identify embedded data	Find filesystems	Firmware image
`if=`	dd input file	Select source	`if=firmware.bin`
`of=`	dd output file	Save carved data	`of=rootfs.img`
`bs=`	dd block size	Byte-accurate copy	`bs=1`

Command/flag	Meaning	When used	Example
`skip=`	Skip input blocks	Start at offset	`skip=670720`
`-t ext2`	Filesystem type	Mount extracted image	Mount command
`-rnw`	Recursive, line, word grep	Locate exact term	Password search
`-e`	grep pattern	Supply search term	`-e password`
`-name`	Match filename	Find config/cert files	`"*.conf"`
`-L`	QEMU library prefix	Match guest rootfs	`-L rootfs`

```
firmware.bin: data
0x9A8: LZMA compressed data
rootfs/etc/passwd
rootfs/etc/shadow
```

`file: data` means no specific known type was identified; Binwalk can still recognize embedded regions. Password/shadow paths reveal account files, not automatically valid credentials. Use `qemu-<architecture>` matching the binary; `mipsel` is little-endian MIPS. Mistakes include wrong offsets, wrong architecture, mounting untrusted images without isolation, and using MD5 as proof of authenticity.

5.10 Nmap industrial-protocol scripts

Purpose: Identify specific OT services with cautious, selected probes.

```
nmap -Pn -sT --scan-delay 1s --max-parallelism 1 \
  -p 502 192.168.20.10
nmap -Pn -sU -p 44818 --script enip-info 192.168.20.10
nmap -Pn -sT -p 46824 192.168.20.20
nmap -Pn -sT -p 1911,4911 --script fox-info 192.168.20.30
nmap -Pn -sT -p 102 --script=s7-info 192.168.20.40
nmap -Pn -sT -p 20547 --script proconos-info 192.168.20.50
nmap -Pn -sT -p 502 --script modbus-discover 192.168.20.60
nmap -Pn -sT -p 9600 --script omron-info 192.168.20.70
nmap -Pn -sU -p 47808 --script bacnet-info 192.168.20.80
nmap -Pn -sT -p 1962 --script pcworx-info 192.168.20.90
```

Flag	Meaning	When used	Example
`-Pn`	Skip host discovery	Device blocks ping	All shown scans
`-sT`	TCP connect scan	No raw SYN privileges	Modbus/S7
`-sU`	UDP scan	UDP OT protocol	EtherNet/IP/BACnet
`--scan-delay`	Wait between probes	Reduce OT load	`1s`

Flag	Meaning	When used	Example
`--max-parallelism`	Limit concurrent probes	Fragile devices	`1`
`--script`	Run NSE script	Extract device details	`s7-info`
`-p`	Select ports	Minimize traffic	`502`

```
PORT    STATE SERVICE
502/tcp open  modbus
| modbus-discover:
|   Unit ID: 1
|_  Device identification: Lab Controller
```

Port state shows reachability; script output identifies protocol details. `Unit ID` selects a Modbus logical slave. The notes show `-P` in several OCR lines; lowercase `-p` is the correct port option. Omron FINS normally uses UDP 9600, while the slide shows `-sT`; treat that exact transport as [UNCLEAR on page 31] and prioritize the port/script pairing in the exam. Active OT scans need approval. Exam focus is matching script, transport, and port.

5.11 Fuzzowski protocol fuzzing

Purpose: Test protocol parsers for crashes and exploitable input-handling errors.

```
python -m fuzzowski 127.0.0.1 47808 -p udp \
  -f bacnet -rt 0.5 -m BACnetMon
python -m fuzzowski 127.0.0.1 502 -p tcp \
  -f modbus -rt 1 -m modbusMon
python -m fuzzowski printer1 631 -f ipp \
  -r get_printer_attribs --restart smartplug
```

Flag	Meaning	When used	Example
`-p`	Transport protocol	TCP, UDP, SSL	`-p udp`
`-f`	Fuzzer/request set	Select protocol	`-f bacnet`
`-rt`	Receive timeout	Bound waiting	`-rt 1`
`-st`	Send timeout	Bound sending	`-st 5`
`-r`	Selected requests	Narrow test	`get_printer_attribs`
`-m`	Monitor module	Observe crashes	`modbusMon`
`--restart`	Restarter module	Recover target	`smartplug`
`--restart-sleep`	Delay after restart	Stabilize target	`5`

```
Target: 127.0.0.1:502
Protocol: tcp
Fuzzer: modbus
Monitor: modbusMon
```

The target/port define the endpoint; protocol and fuzzer must match. Monitor/restart options help detect and recover lab crashes. Upstream Fuzzowski documents IPP fully and BACnet/Modbus as partial implementations; the notes provide the latter commands, so availability can depend on version. The key trap is `-r`: requests, not receive timeout. Never fuzz live production controls.

5.12 Metasploit Modbus auxiliary modules

Purpose: Discover Modbus unit IDs and read register data on an authorized testbed.

```
use auxiliary/scanner/scada/modbus_findunitid
show options
set RHOST 192.168.30.104
run
use auxiliary/scanner/scada/modbusclient
set DATA_ADDRESS 0
set NUMBER 5
set RHOST 192.168.30.104
set UNIT_NUMBER 2
run
```

Option	Meaning	When used	Example
`RHOST`	Target address	Select test PLC	`192.168.30.104`
`RPORT`	Target TCP port	Default Modbus	`502`
`UNIT_ID_FROM`	First unit to test	Discovery range	`1`
`UNIT_ID_TO`	Last unit to test	Discovery range	`254`
`DATA_ADDRESS`	Starting register	Read selected data	`0`
`NUMBER`	Number of values	Limit read size	`5`
`UNIT_NUMBER`	Selected slave	Client request	`2`
`BENICE`	Delay between stations	Reduce load	Seconds, if offered

```
192.168.30.104:502 - correct MODBUS/TCP from stationID 2
192.168.30.104:502 - 5 register values from address 0:
[111, 22, 33, 0, 0]
Auxiliary module execution completed
```

Correct protocol replies identify active station IDs. The array lists five consecutive values beginning at address zero. "Incorrect/none" suggests no usable reply, not conclusive absence. These are auxiliary modules, not necessarily exploit payloads. Exam focus: `modbus_findunitid` discovers; `modbusclient` reads/manipulates data.

5.13 modbus-cli

Purpose: Read, write, and back up Modbus TCP registers and coils.

```
gem install modbus-cli
modbus read 192.168.30.104 %MW100 10
modbus read 192.168.30.104 400101 10
modbus write 192.168.30.104 %MW100 2 2 2 2 2 2 2 2
modbus read 192.168.30.104 %M100 10
modbus read 192.168.30.104 101 10
modbus write 192.168.30.104 %M100 1 1 1 1 1 1 1 1
modbus read --output SCADAregisters.txt \
  192.168.30.104 %MW100 200
modbus read --output SCADAcoils.txt \
  192.168.30.104 %M100 100
```

Item	Meaning	When used	Example
`%MW100`	16-bit memory word	Register access	Read ten words
`%M100`	Boolean memory coil	One-bit access	Read ten coils
`400101`	Register reference	Vendor addressing	Same slide register
`101`	Coil reference	Vendor addressing	Same slide coil
`--word`	Unsigned 16-bit	Default register type	16 bits
`--int`	Signed integer	Negative values	16 bits
`--float`	Floating point	Real values	32 bits
`--dword`	Double word	Larger unsigned data	32 bits
`--output`	Save values	Backup/evidence	Text filename

```
%MW100  1
%MW101  0
%MW102  0
%MW103  17302
%M100   1
%M101   0
```

Each line pairs an address with its value. Registers hold words; coils hold 0 or 1. The final count requests consecutive values, not a single literal value. Writing can change a physical process, so reads are the safer demonstration.

```
:host: 192.168.30.104
:port: 502
:slave: 1
:offset: '101'
:data:
```

The output file records connection metadata, offset, and data for later restoration or comparison. Mistakes include using a coil address for a register, forgetting that float/dword consume 32 bits, and running writes outside a testbed. Exam focus is read versus write, address notation, and `--output`.

5.14 Reading the Fuxnet console evidence

Purpose: Recognize malware-deployment artifacts without treating them as an OT assessment procedure.

```
cat all_sensor_router_ips-ex-civilian.txt
xargs -P10 -I{} ./fuxnet.sh {} 2>logseq
rm fuxnet.sh
```

Part	Meaning	Evidence value
`cat`	Print IP-list contents	Shows selected targets
`xargs`	Build command invocations	Feeds each listed IP
`-P10`	Ten parallel processes	Rapid deployment
`-I{}`	Replace `{}` with input	Per-target argument
`2>logseq`	Redirect standard error	Preserves error trail
`rm`	Remove deployment script	Anti-forensic indicator

The notes also show "Deploying Fuxnet to" lines and JSON fields for IP, port, collector, and control room. These support targeting and execution conclusions. Do not run destructive malware. The exam asks for behavior recognition, not syntax reproduction.

6. Ports, Protocols, Standards and Acronyms

6.1 Core network and OT ports

Protocol	Simple purpose	Default port/ layer	Weakness	Related tool/ attack
MQTT	Lightweight pub/ sub	TCP 1883	Plaintext if no TLS	Wireshark sniffing
Telnet	Remote text terminal	TCP 23	Cleartext credentials	Shodan, default login
SSH	Secure remote terminal	TCP 22	Weak passwords	KmsdBot targeting
HTTP	Web communication	TCP 80	Cleartext without TLS	Camera interface
HTTPS	Encrypted web	TCP 443	Bad certificate checks	Web/API attacks
Modbus/TCP	PLC registers/ coils	TCP 502	No native auth/ encryption	Nmap, modbus-cli
DNP3	Utility SCADA messaging	TCP/UDP 20000	Legacy exposed setups	Shodan remote access
EtherNet/IP	Industrial CIP over IP	TCP/UDP 44818	Exposure and spoofing	Nmap `enip-info`
S7comm	Siemens PLC communication	TCP 102	Unauthorized programming	Nmap `s7-info`
Niagara Fox	Niagara system communication	TCP 1911,4911	Information exposure	Nmap `fox-info`
ProConOS	PLC runtime communication	TCP 20547	Exposed controller data	Nmap script
Omron FINS	Omron PLC communication	UDP 9600 typical	Weak network trust	Nmap `omron-info`
BACnet/IP	Building automation	UDP 47808	Trusting legacy traffic	Nmap/Fuzzowski
PCWorx	Phoenix Contact PLC service	TCP 1962	Controller exposure	Nmap script
HMI slide port	HMI discovery example	TCP 46824	Exposed management	Nmap discovery

Protocol	Simple purpose	Default port/ layer	Weakness	Related tool/ attack
IEC-104	Telecontrol over TCP	TCP 2404	Unauthorized commands	INDUSTROYER.V2
ICCP/TASE.2	Control-center exchange	TCP 102 commonly	Trusted associations	Nessus detection
OPC UA	Industrial interoperability	TCP 4840	Bad endpoint security	IT/OT data exchange
HART-IP	Smart instrument access	TCP/UDP 5094	Field-data exposure	Level 2 discovery
CoAP	Constrained REST messaging	UDP 5683	No security if plain	IoT data access
CoAPS	Secured CoAP	UDP 5684	Bad key management	DTLS protection
LWM2M	IoT device management	UDP 5683/5684	Bootstrap/key flaws	Fleet management
XMPP	Extensible messaging	TCP 5222 client	Weak authentication	IoT messaging
FTP	File transfer	TCP 21	Cleartext credentials	Exposed SCADA banners
SFTP	File transfer over SSH	TCP 22	Weak SSH identity	Secure updates
SMTP	Email delivery	TCP 25	Spoofing/ plaintext	Phishing path
DNS	Name resolution	TCP/UDP 53	Rebinding/ spoofing	Local IoT access
DNSSEC	DNS origin validation	DNS extension	Does not encrypt data	Signed DNS answers
NTP	Time synchronization	UDP 123	Spoofing/ amplification	Log and replay risk
UPnP	Automatic device discovery	UDP 1900	Unwanted port exposure	Disable on routers
Cited malware	Malicious file spread	TCP 48101	Infected-device traffic	Monitor/block

6.2 Wireless, link, fieldbus, and industrial technologies

Name	Simple purpose	Range/layer	Security concern	Example use
BLE	Low-power Bluetooth	Short/link	Pairing and BlueBorne	Wearables
NFC	Very-close communication	Short/link	Relay/ eavesdropping	Access cards
RFID	Radio identification	Short/link	Cloning/tracking	Asset tags
QR/barcode	Optical data labels	Short/visual	Malicious destination	Device onboarding
Wi-Fi	Local IP wireless	Medium/link	Weak keys/rogue AP	Gateways
Wi-Fi Direct	Peer Wi-Fi	Medium/link	Unauthorized pairing	Direct devices
HaLow	Sub-GHz Wi-Fi	Medium/link	Weak configuration	Longer IoT links
LTE-Advanced	Cellular broadband	Long/WAN	Rogue cell/ interception	Mobile gateways
Zigbee	Low-power mesh	802.15.4	Replay/key handling	Sensors
Thread	IPv6 low-power mesh	802.15.4	Commissioning flaws	Smart home
Z-Wave	Low-power mesh	Sub-GHz/link	Pairing/replay	Home control
ANT	Low-power sensor radio	Short/link	Spoofed sensors	Fitness devices
LoRaWAN	Long-range low-power WAN	LPWAN	Key/join weaknesses	Remote meters
Sigfox	Narrowband LPWAN	LPWAN	Limited payload security	Simple sensors
Neul	Early LPWAN technology	LPWAN	Legacy deployments	Wide-area IoT
NB-IoT	Cellular narrowband IoT	LPWAN	Core/device trust	Utility sensors
6LoWPAN	IPv6 over constrained links	Adaptation	Fragmentation/ trust	802.15.4 IP
VSAT	Satellite terminal link	Long/WAN	Remote-link exposure	Remote sites

Name	Simple purpose	Range/layer	Security concern	Example use
Li-Fi	Optical wireless	Short/link	Proximity interception	Optical data
Ethernet	Wired local networking	Link	Sniffing if unencrypted	Plant backbone
MoCA	Networking over coax	Link	Unauthorized coax access	Building links
Power-line PLC	Data over power wiring	Link	Shared-medium exposure	Building IoT
Profibus	Industrial fieldbus	Fieldbus	No inherent modern auth	Field devices
Profinet	Industrial Ethernet	L2/IP based	Unauthorized control	Controllers
ControlNet	Deterministic industrial net	Fieldbus	Legacy trust	PLC networks
DeviceNet	Industrial device network	CAN based	Legacy trust	Sensors/actuators
CANopen	CAN application protocol	CAN based	Spoofed messages	Embedded control
EtherCAT	Real-time industrial Ethernet	L2	Timing/control abuse	Motion control
Sercos II	Real-time motion bus	Fieldbus	Legacy trust	Drives
FOUNDATION Fieldbus	Digital process fieldbus	Fieldbus	Unauthorized field access	Process instruments
AS-Interface	Simple actuator/ sensor bus	Fieldbus	Physical trust	Level 2 devices
CC-Link	Industrial control network	Fieldbus/Ethernet	Controller trust	Factory automation
HART	Smart analog/ digital instruments	Field protocol	Unauthorized config	Process sensors
MELSEC-Q	Mitsubishi PLC family/protocol	Controller	Programming exposure	Level 0/1 PLC
GE-SRTP	GE service request protocol	Industrial	Controller exposure	GE PLC communication

Name	Simple purpose	Range/layer	Security concern	Example use
Crimson v3	Red Lion configuration software	OT software	Project/file risk	HMI configuration
SuiteLink	Wonderware data exchange	Industrial service	Trusted data access	Operations data
FTE	Fault Tolerant Ethernet	Industrial network	Trusted control traffic	Honeywell systems
HSCP	OT protocol listed in notes	Industrial	Verify exact expansion	Purdue Level 3

[UNCLEAR on page 22] HSCP is reproduced as printed, but its expansion is not reliably established from the notes. Do not invent a port for it. "Edge" and "Physical Web" are application technologies, while Mihini/M3DA is a machine-to-machine messaging framework; no single exam default port should be inferred.

> **ADDED CONTEXT:** Modbus function 03 reads holding registers and 04 reads input registers; 01 reads coils; 06 writes one holding register. Coils/discrete values are one bit, while registers are 16 bits. Exact support varies by device.

6.3 Standards and architecture terms

Standard/term	Simple purpose	Where used	Security concern	Related defense
ISA/IEC 62443	Industrial security standard	All OT zones	Poor role/ requirement mapping	Zones and conduits
IEC 61850	Power-substation communication	Utility OT	Engineering/ access abuse	Segmentation/ monitoring
IEC 60870-5-101	Serial telecontrol	Utility field	Legacy trust	Controlled gateways
ISA SP100	Wireless industrial standard	Field wireless	Key/network trust	Secure commissioning
PERA/Purdue	ICS architecture model	IT/OT	Flat direct connections	IDMZ and zoning
MITRE ATT&CK ICS	Adversary behavior knowledge	Detection/ response	Missing coverage	Behavior detections

Standard/term	Simple purpose	Where used	Security concern	Related defense
OWASP IoT Top 10	Common IoT weakness list	Design/review	Misclassification	Secure lifecycle
PKI	Certificates and key trust	Identity/ encryption	Bad revocation checks	Managed CA/CRL
TLS	Transport encryption	TCP applications	Old versions/bad validation	Modern TLS/mTLS
SSLv2	Obsolete secure transport	Legacy devices	Broken security	Disable immediately
IPsec	IP-layer protection	Network/VPN	Weak policy/keys	Encrypted conduits
VPN	Protected remote tunnel	Remote access	Broad post-login access	MFA plus least privilege
RBAC	Role-based access control	Users/vendors	Excessive roles	Role review
DLP	Data-loss prevention	Enterprise	Sensor/data exfiltration	Egress controls
IDS/IPS	Detect/prevent intrusions	IT/OT monitoring	Unsafe active response	Passive OT tuning
DMZ/IDMZ	Buffer between trust zones	IT/OT boundary	Direct bypass paths	Brokered services
Zero trust	Verify and minimize access	Legacy/modern OT	Implicit vendor trust	Continuous validation
CRL	Certificate revocation list	PKI	Stale revocation data	Current validation
FOTA	Firmware over the air	IoT updates	Unsigned/ malicious image	Signed secure update
RSSI	Received signal strength	RF testing	Signal filtering	RFCrack bounds
BSSID	Wireless AP radio identity	Wi-Fi recon	Device identification	Local discovery

6.4 Quick acronym decoder

Acronym	Meaning	One-line role
IoT/IoE	Internet of Things/Everything	Connected sensing objects
IIoT	Industrial Internet of Things	IoT in industry

Acronym	Meaning	One-line role
OT/ICS	Operational Technology/Control System	Physical industrial control
SCADA	Supervisory Control and Data Acquisition	Wide-area supervision
PLC	Programmable Logic Controller	Local control logic
RTU	Remote Terminal Unit	Remote field interface
DCS	Distributed Control System	Distributed plant control
HMI	Human-Machine Interface	Operator display/control
BPCS	Basic Process Control System	Routine process control
SIS	Safety Instrumented System	Protective safe-state control
IED	Intelligent Electronic Device	Smart protection/monitoring
VFD	Variable-Frequency Drive	Motor-speed control
PID	Proportional-Integral-Derivative	Feedback control algorithm
SDR	Software-Defined Radio	Programmable radio
EMFI/BBI	Electromagnetic/Body-Bias Injection	Hardware fault methods
UART/SPI/I2C	Serial hardware interfaces	Board communication
JTAG	Joint Test Action Group	Hardware test/debug
C2/C&C	Command and Control	Botnet tasking channel
NSE	Nmap Scripting Engine	Nmap service scripts
NSM	Network Security Monitoring	Traffic-based visibility

7. Attack vs Countermeasure and Look-alike Comparisons

7.1 Attacks and best-matched countermeasures

Attack	How to recognize	Primary countermeasure	Supporting control	Avoid this mistake
IoT DDoS	Many bots flood target	Upstream DDoS protection	Patch and egress filter	Only rebooting server
Rolling code	Jam, capture, reuse	Authenticated freshness	Failure alerts	Plain replay defense
Jamming	Same-frequency interference	RF monitoring/ resilience	Alternate channels	Encryption only
Backdoor phishing	Attachment creates access	Email/endpoint controls	MFA and reporting	Firewall only
Telnet access	TCP 23 and weak login	Disable Telnet/use SSH	MFA and segmentation	Hiding port only
DNS rebinding	Hostname changes to private IP	Host/DNS validation	Local interface auth	Ordinary antivirus only
Replay	Valid old message resent	Nonce/timestamp freshness	Message authentication	Encryption alone
Fault injection	Glitch bypasses checks	Secure boot/glitch sensors	Tamper resistance	Software patch only
Side channel	Timing/power reveals secret	Constant-time/ leak resistance	Physical shielding	Longer key only
Firmware persistence	Modified image survives	Signed secure boot	Trusted reflash	Password reset only
HMI memory attack	Overflow/project exploit	Patch and input validation	Remove unsafe ActiveX	Only stronger password
PLC rootkit	Hidden I/O manipulation	Logic integrity verification	Restrict programming	Trusting HMI display
Evil PLC	PLC attacks workstation	Workstation/ controller isolation	Signed project files	Treat PLC as trusted
RF re-pairing	Hostile controller paired	Authenticated pairing	Physical control	Replay detection only

Attack	How to recognize	Primary countermeasure	Supporting control	Avoid this mistake
OT supply chain	Trusted vendor is path	Supplier verification	Signed updates/ RBAC	Blind vendor trust
Modbus manipulation	Unauthorized register write	Zone/ACL master restriction	Function-code monitoring	Flat OT network
Outbound OT C2	Controller reaches internet	Egress deny by default	Separate update broker	Inbound rules only

7.2 Look-alike pairs

X vs Y	Difference in one line	Question clue
IoT vs OT	Connected things vs industrial physical control	Camera vs production PLC
PLC vs RTU	Local logic vs remote telemetry/control	Deterministic machine vs remote site
SCADA vs DCS	Wide supervision vs distributed plant control	Pipeline vs processing plant
HMI vs SCADA	Operator screen vs supervisory system	Display vs whole architecture
Level 0 vs Level 1	Physical process vs basic controllers	Sensor/motor vs PLC
DMZ vs IDMZ	General buffer vs industrial IT/ OT buffer	Purdue Level 3.5
Shodan vs Nmap	Indexed banners vs active probes	Search engine vs scanner
Nmap vs Nessus	Service discovery vs known-flaw assessment	Open port vs vulnerability
Wireshark vs Malcolm	Packet decoder vs full NSM platform	Frame details vs dashboards
Jamming vs replay	Interference vs retransmission	Noise vs old command
Replay vs rolling-code attack	Reuse old code vs jam and save unused code	Two button presses
MITM vs side channel	Intercept communication vs observe leakage	Proxy vs oscilloscope
Fault vs side channel	Disturb behavior vs measure behavior	Voltage glitch vs timing

X vs Y	Difference in one line	Question clue
UART vs JTAG	Serial console vs debug/test control	RX/TX/baud vs execution
Binwalk vs QEMU	Extract firmware vs emulate execution	Squashfs vs MIPS/ARM
Register vs coil	Multi-bit storage vs one-bit switch	`%MW` vs `%M`
Evil PLC vs PLC rootkit	PLC attacks workstation vs hidden I/O control	Upload procedure vs rootkit
Impair vs inhibit	Manipulate process vs suppress response	Setpoints vs blocked alarms
Initial access vs persistence	Enter environment vs remain after restart	Phishing vs firmware
Encryption vs authentication	Hide content vs prove identity	Confidentiality vs trusted sender
Secure boot vs signed update	Verify startup chain vs verify new image	Boot trust vs installation
IT patch vs OT patch	Routine update vs tested safe change	Maintenance window clue

8. Exam Cheat Sheet

8.1 Keyword -> answer

Keyword in question	Best answer
Internet-connected camera banners	Shodan
Factory-set camera passwords	IoTSeeker or CIRT.net
IoT paths and versions	Genzai
1 MHz to 6 GHz half duplex	HackRF One
YardStick rolling code `-r`	RFCrack
802.15.4 frame capture/replay	Open Sniffer or KillerBee
RX/TX and baud 9600	UART/BUS Auditor
SCL and SDA	I2C
Debug execution flow	JTAG/GDB/OpenOCD
Ground flash I/O during boot	NAND glitching
Extract Squashfs	Binwalk/Firmware Mod Kit
Emulate MIPS firmware	QEMU
Camera `-a` single address	CamOver
MQTT ping/publish TCP 1883	MQTT broker traffic
OpenSearch plus Arkime	Malcolm
Known SCADA vulnerabilities	Nessus with SCADA plugins
BACnet, Modbus malformed requests	Fuzzowski
IT/OT attack-path prioritization	Skybox
Unit identifier discovery	`modbus_findunitid`
Read `%MW100`	modbus-cli register read
Physical process	Purdue Level 0
Operations/site operations	Purdue Level 3
IT/OT broker boundary	IDMZ Level 3.5
Blocked alarm/report messages	Inhibit response function
Altered parameters/I/O	Impair process control
Sensor gateway destruction	Fuxnet

Keyword in question	Best answer
Oscilloscope password inference	Power side channel
Safest first OT discovery step	Passive traffic/asset review

8.2 Must-memorize lists and numbers

1. IoT architecture: Edge, Access Gateway, Internet, Middleware, Application.
2. Purdue: L0 process, L1 basic control, L2 supervisory, L3 operations, L3.5 IDMZ, L4 logistics, L5 enterprise.
3. Hacking flow in this module: information gathering, vulnerability scanning, launch attacks, gain remote access, maintain access, then countermeasures.
4. Modbus 502/TCP; DNP3 20000; S7 102/TCP; EtherNet/IP 44818; BACnet 47808/UDP; Niagara Fox 1911/4911; ProConOS 20547; Omron FINS 9600; PCWorx 1962.
5. Telnet 23/TCP; SSH 22/TCP; MQTT 1883/TCP; CoAP 5683/UDP; IPP 631/TCP.
6. RFCrack: `-i` live replay, `-r` rolling, `-j` jam, `-M` modulation, `-F` frequency, `-U/-L` RSSI bounds.
7. HackRF: `-r` receive, `-t` transmit, `-f` frequency, `-x` TX VGA gain.
8. Nmap OT: `-Pn` no ping, `-sT` TCP connect, `-sU` UDP, `-p` ports, `--script` NSE, slow delay and parallelism one for fragile devices.
9. Zero trust: define, map, architect, policy, monitor/maintain.
10. MITRE ICS effect: inhibit response blocks response; impair process changes control; impact damages/disrupts/denies.

8.3 Pick the BEST option rules

- If two answers find flaws, choose passive asset/traffic review before active OT scans.
- If the question asks "which tool", match its unique verb: search, scan, decode, fuzz, emulate, extract, debug, or manage.
- If a controller is live and safety matters, do not choose fuzzing or write tests.
- If credentials are default, replace them with unique strong credentials before cosmetic actions such as changing banners.
- If IT and OT connect directly, choose IDMZ/segmentation rather than only antivirus.
- If malware persists in firmware, choose trusted reflash and secure boot, not a password reset.
- If the question describes two fob presses, jamming, and a saved code, choose rolling-code attack over simple replay.
- If a port is open, do not assume exploitation; validate safely and consider banner ambiguity.

- If one control solves root cause and another only detects, choose root-cause fixing unless the scenario makes immediate change unsafe.
- Use commands only on authorized systems; in the exam, operational safety and scope are part of the best answer.

9. Practice Questions

Question 1: IoT architecture

A sensor sends a Zigbee message to a local gateway. The gateway converts it into an IP message. Which IoT architecture layer performs this conversion?

A. Edge technology B. Access gateway C. Middleware D. Application

Answer: B. Access gateway. Protocol translation and messaging belong to this layer. A is wrong because edge contains sensors/devices. C is wrong because middleware manages devices/information. D is wrong because applications deliver user functions.

Question 2: Rolling-code scenario

During an authorized vehicle test, the first fob press fails. The test device forwards the first captured code after the second press, unlocks the car, and retains the second code. What is being demonstrated?

A. DNS rebinding B. Simple sniffing only C. Rolling-code attack D. Power-analysis attack

Answer: C. Rolling-code attack. Jamming/capture and saving an unused later code are the distinctive sequence. A changes DNS answers. B does not explain forwarding and jamming. D measures chip power rather than RF command codes.

Question 3: Best first OT step

A consultant receives permission to inventory an old water-treatment network. Several PLCs stop if exposed to heavy traffic. What is the BEST first step?

A. Fuzz Modbus on every controller B. Run maximum-speed parallel scans C. Review passive traffic and existing asset records D. Write test values to unused coils

Answer: C. Review passive traffic and existing asset records. It builds visibility with minimal process risk. A and B can crash fragile systems. D can change a physical process and is not inventory gathering.

Question 4: Nmap flag reading

What does `--max-parallelism 1` do in the OT Nmap command shown in the notes?

A. Restricts the scan to one port B. Limits concurrent probes to one C. Selects one target host D. Runs only one NSE script

Answer: B. Limits concurrent probes to one. It reduces traffic pressure. A is controlled with `-p`. C is determined by the target argument. D is controlled by the `--script` selection.

Question 5: OT script and protocol

Which command is the best match for identifying a Siemens SIMATIC S7 PLC?

A. `nmap -Pn -sU -p 47808 --script bacnet-info TARGET` B. `nmap -Pn -sT -p 102 --script=s7-info TARGET` C. `nmap -Pn -sT -p 502 --script modbus-discover TARGET` D. `nmap -Pn -sU -p 44818 --script enip-info TARGET`

Answer: B. S7comm uses TCP 102 and the `s7-info` NSE script. A is BACnet/IP. C is Modbus/TCP. D is EtherNet/IP discovery.

Question 6: Firmware analysis

An assessor extracts a firmware image and finds a Squashfs filesystem. The next goal is to run one MIPS binary with its extracted libraries. Which tool fits BEST?

A. QEMU B. Shodan C. Nessus D. CIRT.net

Answer: A. QEMU. It emulates the required processor environment. B searches internet exposure. C finds known vulnerabilities. D supplies default-password data.

Question 7: Hardware interface

A BUS Auditor report lists device address 0x48, SCL pin 1, and SDA pin 0. Which interface was identified?

A. UART B. I2C C. JTAG D. SPI

Answer: B. I2C. SCL and SDA are its clock and data lines. UART instead reports RX/TX and baud. JTAG is a debug/test interface. SPI uses a different signal arrangement.

Question 8: RFCrack flag

Which RFCrack command selects jamming at the specified frequency?

A. `python RFCrack.py -i` B. `python RFCrack.py -r -M MOD_2FSK -F 314350000` C. `python RFCrack.py -j -F 314000000` D. `python RFCrack.py -U -100 -L -10`

Answer: C. `-j` is jamming and `-F` sets frequency. A is live replay. B is rolling-code mode. D supplies RSSI bounds but no jamming selection.

Question 9: Purdue placement

Which component is placed at Purdue Level 0?

A. Business logistics database B. Site historian C. Area supervisory HMI D. Physical temperature sensor

Answer: D. Physical temperature sensor. Level 0 is the physical process. A is Level 4. B belongs to site operations at Level 3. C is representative of Level 2.

Question 10: MITRE ATT&CK for ICS

Malware prevents an HMI from receiving alarm reports, while leaving the underlying process parameters unchanged. Which tactic fits BEST?

A. Inhibit response function B. Initial access C. Collection D. Privilege escalation

Answer: A. Inhibit response function. Blocking reporting thwarts response to a hazard. B describes entry. C describes gathering data. D describes gaining greater privileges, not suppressing alarms.

Question 11: Evil PLC

A compromised PLC's logic is changed. Later, an engineering workstation connects and initiates an upload procedure that exposes it to the hostile PLC. What is this called?

A. Evil PLC attack B. Sybil attack C. DNS rebinding D. NAND glitching

Answer: A. Evil PLC attack. The controller is weaponized against workstations. B uses many forged identities. C changes hostname resolution. D disturbs flash during boot to seek root access.

Question 12: AI-assisted assessment

An analyst wants to use a public generative-AI service to help classify an authorized Malcolm protocol inventory. What is the BEST approach?

A. Upload credentials and raw customer captures for maximum accuracy B. Ask it to generate live PLC write commands C. Provide minimized non-sensitive records and require evidence for each mapping D. Let it decide whether production fuzzing is safe

Answer: C. Minimization and evidence-based mappings reduce disclosure and hallucination risk. A exposes sensitive material. B can endanger a process. D delegates a safety decision that requires asset owners and operational evidence.

Question 13: Modbus command reading

In `modbus read 192.168.30.104 %MW100 10`, what does the final `10` request?

A. Ten consecutive register values B. A ten-bit register C. Unit identifier ten only D. Ten seconds of packet capture

Answer: A. Ten consecutive register values. The address selects the start and the count selects consecutive values. B confuses count with width. C is the slave/unit, not this argument. D is unrelated to this Modbus read syntax.

Question 14: Metasploit module selection

Which module should an authorized tester use first to discover active Modbus station identifiers?

A. `auxiliary/scanner/scada/modbus_findunitid` B. `auxiliary/scanner/scada/modbusclient` C. `camover` D. `fuzzowski`

Answer: A. Its purpose is unit-ID discovery. B reads/manipulates data after a unit is selected. C targets camera password disclosure. D tests protocol handling with malformed/varied inputs.

Question 15: HackRF flags

What is the purpose of `hackrf_transfer -r capture.raw -f 433900000`?

A. Transmit a file at 433.9 kHz B. Receive samples to a file at 433.9 MHz C. Display HackRF firmware information D. Jam all nearby 433 MHz devices

Answer: B. `-r` receives, and frequency is expressed in Hz. A reverses direction and misreads the units. C uses `hackrf_info`. D describes neither the receive flag nor a controlled replay capture.

Question 16: Default-credential scenario

A building contractor can reach every camera's administration page from the internet, and all retain one factory password. What is the BEST combined response?

A. Change only the web page title B. Enable UPnP so cameras can manage exposure C. Restrict management exposure and assign unique strong credentials D. Leave credentials because network scans found no CVE

Answer: C. It removes both exposure and shared-secret risk. A is cosmetic. B can increase unwanted exposure. D confuses lack of a scanner result with secure settings.

Question 17: Firmware persistence

A device remains compromised after its administrator password is changed. Analysis shows an unsigned modified firmware image. Which defense addresses the root cause BEST?

A. Signed firmware with secure boot and trusted reflash B. A longer administrator password only C. Disable wireless SSID broadcast D. Increase log font size

Answer: A. It verifies the boot/update chain and removes the implanted image. B does not remove firmware persistence. C does not provide meaningful firmware integrity. D is unrelated.

Question 18: Fuzzing tool

Which statement BEST distinguishes Fuzzowski from Nessus in this module?

A. Fuzzowski tests malformed protocol inputs; Nessus maps known vulnerabilities B. Fuzzowski only searches Shodan; Nessus only emulates firmware C. Fuzzowski manages updates; Nessus pairs RF remotes D. They are identical packet-capture programs

Answer: A. Robustness fuzzing differs from known-vulnerability assessment. B describes neither tool correctly. C describes unrelated functions. D ignores their different assessment methods.

Question 19: OT malware recognition

An incident involves destructive ICS malware that modifies data, blocks sensor-gateway access, and attempts to damage sensors. Which family matches the notes?

A. Fuxnet B. KmsdBot C. CamOver D. RFCrack

Answer: A. Fuxnet. Those are its described behaviors. B is IoT botnet malware. C and D are assessment tools, not the listed OT malware family.

Question 20: Segmentation scenario

A compromised finance workstation can connect directly to PLCs because corporate IT and manufacturing share one flat network. Which design improvement is BEST?

A. Move all PLCs onto the enterprise wireless network B. Establish an IDMZ and allow only required brokered flows C. Give every contractor the same OT administrator account D. Permit unrestricted outbound access for controller updates

Answer: B. Zones and an IDMZ limit direct trust and required pathways. A increases exposure. C violates least privilege and accountability. D increases malware/C2 risk.

Question 21: AI attack angle

A vendor receives a highly fluent, personalized email and a realistic voice message asking for an emergency OT VPN exception. What is the BEST immediate action?

A. Trust it because the grammar and voice are accurate B. Approve broad permanent access to save time C. Verify through a known independent channel and follow the access-approval process D. Upload internal network diagrams to a public chatbot for advice

Answer: C. AI phishing/deepfakes make content quality a weak trust signal; verified procedure is the control. A and B reward impersonation. D adds data exposure and does not authenticate the request.

Question 22: Supply-chain recognition

A maintenance contractor's account is stolen and its legitimate remote-support channel is used to reach the OT network. Which attack category fits BEST?

A. Service-provider supply-chain compromise B. Optical fault injection C. Zigbee beacon replay only D. Username enumeration through a login form

Answer: A. A trusted service provider's access is the infiltration path. B is hardware glitching. C requires captured Zigbee frames. D concerns valid-account-name discovery, not abuse of a supplier relationship.

Question 23: Command output interpretation

Metasploit prints `5 register values from address 0: [111, 22, 33, 0, 0]`. What can the tester conclude?

A. Five Modbus values were read beginning at address zero B. Five cameras still use default passwords C. Five stations were proven absent D. The PLC was safely patched

Answer: A. The output explicitly reports the count, starting address, and values. B belongs to a camera scanner. C confuses data read with unit discovery. D does not follow from a successful read.

Question 24: Best vulnerability-scanning decision

A Nessus demonstration policy scans ports 0-1000, but the analyst needs Modbus, S7, EtherNet/IP, and BACnet coverage. What is the BEST observation?

A. The shown range omits several required OT ports and needs deliberate adjustment B. Port 502 proves all industrial services are covered C. Nessus cannot ever assess SCADA systems D. A completed scan with no SCADA plugins proves there are no SCADA flaws

Answer: A. Modbus 502 is included, but S7 102 is too while 44818 and 47808 are outside 0-1000; policy/plugins must match scope. B is overbroad. C is false because SCADA plugins are shown. D confuses missing coverage with absence of vulnerabilities.

10. Flashcards

Q: What are the five IoT architecture layers? | A: Edge, access gateway, internet, middleware, application. Q: Which IoT layer translates protocols? | A: Access gateway layer. Q: Which IoT layer manages devices and information? | A: Middleware layer. Q: What does MQTT normally use? | A: Publish/subscribe messaging through a broker, normally TCP 1883. Q: What is the classic Modbus/TCP security weakness? | A: No built-in authentication or encryption. Q: What is the Modbus/TCP default port? | A: TCP 502. Q: What is the DNP3 default port? | A: TCP or UDP 20000. Q: Which port and script identify Siemens S7? | A: TCP 102 with Nmap `s7-info`. Q: Which port and script identify EtherNet/IP? | A: UDP or TCP 44818; notes use UDP with `enip-info`. Q: Which port and script identify BACnet? | A: UDP 47808 with `bacnet-info`. Q: Which ports identify Niagara Fox? | A: TCP 1911 and 4911 with `fox-info`. Q: What does Shodan provide? | A: Indexed internet-exposure data such as IP, port, and banner. Q: What does CIRT.net provide in this module? | A: A database of factory/default device passwords. Q: What is the rolling-code sequence? | A: Jam/capture first, jam/capture second, forward first, save second. Q: What does RFCrack `-r` select? | A: Rolling-code mode. Q: What does RFCrack `-j` select? | A: Jamming mode. Q: What do HackRF `-r` and `-t` mean? | A: Receive to a file and transmit from a file. Q: What is HackRF One's approximate range and duplex? | A: 1 MHz to 6 GHz, half duplex. Q: Which tool captures and replays 802.15.4 frames? | A: Open Sniffer; KillerBee is a close Zigbee alternative. Q: Which interface uses RX, TX, and baud rate? | A: UART. Q: Which interface uses SCL and SDA? | A: I2C. Q: Which interface is most associated with debugging execution? | A: JTAG. Q: What is NAND glitching? | A: Disturbing flash serial I/O during boot to seek privileged access. Q: What does CamOver `-a` specify? | A: One target camera address. Q: What does Firmware Mod Kit do? | A: Deconstructs and reconstructs firmware images. Q: What does Binwalk do? | A: Identifies and extracts embedded firmware data. Q: What does QEMU do in firmware assessment? | A: Emulates a binary or system architecture. Q: What is a PLC? | A: A controller executing local logic for a physical process. Q: What is an RTU? | A: A remote field telemetry/control unit. Q: What is the difference between SCADA and HMI? | A: SCADA supervises systems; HMI is the operator interface. Q: What is Purdue Level 0? | A: The physical process. Q: What is Purdue Level 3.5? | A: The industrial DMZ between IT and OT. Q: What is the safest initial discovery approach in fragile OT? | A: Passive traffic and asset-record review. Q: Which MITRE tactic blocks alarms or command reports? | A: Inhibit response function. Q: Which MITRE tactic alters parameters or I/O control? | A: Impair process control. Q: What is an Evil PLC attack? | A: A hostile PLC is used to attack connected engineering workstations. Q: What does Metasploit `modbus_findunitid` do? | A: Discovers responding Modbus unit identifiers. Q: What does `%MW100` represent in modbus-cli examples? | A: A 16-bit register/memory-word address. Q: What does `%M100` represent in modbus-cli examples? | A: A one-bit coil address. Q: What are the five zero-trust steps in the notes? | A: Define, map, architect, policy, monitor/maintain.

11. Glossary

- **Access gateway:** IoT layer that translates protocols and handles messaging.
- **Active scanning:** Sending probes to a target to discover services or weaknesses.
- **Actuator:** Device that changes a physical process, such as a valve or motor.
- **Asset register:** Maintained inventory of systems, owners, versions, and criticality.
- **Backdoor:** Unauthorized or hidden method of bypassing normal access controls.
- **BACnet:** Building-automation protocol, commonly carried over UDP 47808.
- **Baud rate:** Signaling speed configuration for a serial interface such as UART.
- **Binwalk:** Tool that identifies and extracts embedded firmware content.
- **BSSID:** Identifier of a wireless access-point radio.
- **Botnet:** Collection of compromised devices controlled by an attacker.
- **C2:** Command-and-control channel used to task compromised systems.
- **CoAP:** REST-like application protocol for constrained IoT devices.
- **Coil:** One-bit Modbus value that can represent a controllable switch state.
- **Conduit:** Controlled communication path between OT security zones.
- **DCS:** Distributed control system used to coordinate plant processes.
- **Deepfake:** AI-generated or altered media that imitates a real person or event.
- **DMZ:** Buffer network separating zones with different trust levels.
- **DNP3:** Industrial protocol commonly used by utility SCADA systems.
- **DNS rebinding:** Changing DNS answers to make browser script reach another address.
- **Edge layer:** IoT devices, sensors, machines, and intelligent edge nodes.
- **EMFI:** Electromagnetic fault injection used to disturb chip behavior.
- **EtherNet/IP:** Industrial protocol carrying CIP over standard IP networks.
- **Evil PLC:** Weaponized controller used to attack engineering workstations.
- **Fault injection:** Deliberate disturbance of operating conditions to alter execution.
- **Firmware:** Low-level software that controls a hardware device.
- **FOTA:** Firmware-over-the-air update mechanism.
- **Fuzzing:** Sending varied or malformed inputs to discover handling failures.
- **Gateway:** System that connects networks and may translate protocols.
- **Half duplex:** Ability to transmit or receive, but not both simultaneously.
- **HMI:** Human-machine interface for monitoring and operator commands.
- **HVAC:** Heating, ventilation, and air-conditioning systems.

- **I2C:** Addressed hardware bus using SCL and SDA.
- **ICS:** Industrial control system and its associated equipment and networks.
- **IDMZ:** Industrial demilitarized zone between enterprise and manufacturing zones.
- **IIoT:** Internet of Things applied to industrial operations.
- **JTAG:** Hardware test/debug interface that can expose memory and execution.
- **Jamming:** Radio interference that prevents reliable communication.
- **Malcolm:** Network security monitoring platform with ICS protocol visibility.
- **Middleware:** IoT layer providing device and information management.
- **MITM:** Man-in-the-middle interception between two communicating parties.
- **Modbus:** Simple industrial protocol for registers, coils, and device control.
- **Monitor mode:** Wireless adapter mode for capturing radio frames.
- **MQTT:** Lightweight publish/subscribe messaging protocol using a broker.
- **Nessus:** Vulnerability-assessment product with SCADA-related plugins.
- **NSE:** Nmap Scripting Engine used for service-specific discovery scripts.
- **OT:** Operational technology that monitors or controls industrial physical devices.
- **Passive discovery:** Learning assets from existing records or observed traffic.
- **Purdue model:** Layered architecture separating enterprise, operations, and process.
- **PLC:** Programmable logic controller executing deterministic control logic.
- **Power analysis:** Side channel that infers secrets from electrical consumption.
- **Purdue IDMZ:** Level 3.5 boundary used to broker IT/OT communication.
- **QEMU:** Emulator used to run firmware binaries for another processor architecture.
- **Ransomware:** Malware that denies data or device access and demands payment.
- **Register:** Multi-bit Modbus storage value, commonly 16 bits.
- **Replay:** Retransmission of previously captured valid communication.
- **RF:** Radio frequency communication.
- **Rolling code:** Changing authentication code sequence used by key fobs.
- **Root of trust:** Trusted hardware/software anchor for verifying system integrity.
- **Rootkit:** Malware that hides itself and maintains privileged control.
- **RSSI:** Received signal strength indicator.
- **RTU:** Remote terminal unit connecting distant field equipment to supervision.
- **SCADA:** Supervisory control and data acquisition across industrial processes.
- **SDR:** Software-defined radio with programmable signal processing.
- **Secure boot:** Startup chain that verifies trusted software before execution.
- **Segmentation:** Dividing a network into controlled security zones.

- **Sensor:** Device that measures a physical property.
- **Shodan:** Search engine indexing internet-facing device and service information.
- **Side channel:** Information leakage through timing, power, emissions, or behavior.
- **SIS:** Safety instrumented system designed to bring a process to a safe state.
- **SPI:** Synchronous serial interface for chips and peripherals.
- **Squashfs:** Compressed read-only filesystem often found in embedded firmware.
- **Sybil attack:** Use of many forged identities to manipulate a network.
- **Telnet:** Plaintext remote-terminal protocol using TCP 23.
- **Timing analysis:** Side channel that infers secrets from operation duration.
- **TLS:** Transport Layer Security used to protect application communication.
- **UART:** Asynchronous serial hardware interface using receive/transmit lines.
- **UPnP:** Automatic discovery/configuration technology often disabled on routers.
- **VPN:** Protected network tunnel used for managed remote access.
- **Zero trust:** Security model that continuously verifies and minimizes access.
- **Zigbee:** Low-power IEEE 802.15.4 mesh networking technology.

12. Coverage Report

12.1 Page-to-section map

Page	Source topics	Guide sections
1	Objectives, IoT definition, data flow	1, 3.1
2	IoT layers, technologies	3.2
3	Protocol repeat, OWASP Top 10	3.2, 3.3
4	Attack surfaces, vulnerabilities	3.3
5	Threat list, DDoS	3.4
6	HVAC, rolling code	3.4, 3.5
7	Jamming, smart-grid backdoor	3.5, 3.6
8	SDR replay/cryptanalysis/recon	3.7
9	Local discovery, rebinding, faults	3.7, 3.8
10	Other attacks, IoT malware	3.8, 3.9
11	Shodan, wireless sniffing	3.10, 5.1-5.2
12	IoTSeeker, Genzai	3.11, 5.3
13	RFCrack, Open Sniffer	3.12, 5.4
14	HackRF One and replay	3.12-3.13, 5.5
15	Buses, NAND glitching	3.13-3.14, 5.6
16	CamOver, Telnet	3.14, 5.7
17	Firmware access and analysis	3.15-3.16, 5.8-5.9
18	IoT tools, defenses	3.17-3.18
19	Manufacturer/OWASP solutions	3.18
20	Hardware and device management	3.17-3.18
21	OT, ICS, convergence	3.19
22	Purdue and OT protocols	3.20, 6.2
23	OT challenges/vulnerabilities	3.21
24	MITRE ATT&CK for ICS	3.22

Page	Source topics	Guide sections
25	OT threats, HMI	3.23
26	HMI example, side channels	3.23
27	PLC rootkit, Evil PLC	3.24
28	RF remote controllers	3.24
29	Supply chain, OT malware	3.24-3.25, 5.14
30	Shodan OT, CIRT.net	3.26
31	Nmap OT, Modbus Wireshark	3.27, 5.10
32	Malcolm, Nessus	3.27-3.28
33	Fuzzowski, Skybox	3.28, 5.11
34	Hardware, Metasploit	3.29, 5.12
35	modbus-cli types/read/write	3.29, 5.13
36	Modbus output, DNP3	3.29, 5.13
37	OT defense and solutions	3.30
38	Purdue controls, zero trust	3.30
39	Organizations, module summary	3.30, 1

12.2 Unclear items and best readings

- [UNCLEAR on page 11] OCR shows `nmap -P` and `airodump-ng start wlan0mon`. The guide uses standard `-p` and `airodump-ng wlan0mon`, labelled as corrections.
- [UNCLEAR on page 12] The small Genzai screenshot's CVE identifier is not reliable; the guide keeps the confirmed TP-Link/default-password concept and omits the number.
- [UNCLEAR on page 14] HackRF raw capture filenames are blurry. `capture.raw` is a clearly labelled teaching substitute; flags and frequency follow the notes.
- [UNCLEAR on page 22] HSCP appears in the Purdue protocol list, but its expansion cannot be confirmed from the supplied text. It is retained without an invented port.
- [UNCLEAR on page 29] Parts of Fuxnet JSON UUID values are illegible. The guide covers the legible structure and behavior without reconstructing identifiers.
- [UNCLEAR on page 31] The Omron example shows TCP while FINS commonly uses UDP 9600. Both the slide syntax and the transport conflict are explicitly identified.

12.3 Items to double check

- Tool versions change. Fuzzowski's installed build may not include every protocol fuzzer or monitor shown in the course; upstream documentation notes partial support.
- RFCrack RSSI `-U/-L` behavior is tool-specific. Memorize the notes' example rather than generalizing the numeric ordering to other RF tools.
- Port numbers identify common defaults, not proof of service or device identity.
- Shodan coordinates and country/city results can be inaccurate; verify ownership before any active test.
- The supplied attachment is a page-text transcription, not the original images. Blurry small-text items above are not guessed silently.

12.4 Coverage confirmation

All 39 inventory lines are mapped above. Every named tool, attack, protocol, command, flag, defense, standard, organization, and legible diagram sequence is covered in exactly one primary topic section and cross-referenced in the reference sections. Repeated screenshots are merged without dropping unique details. No real exam dumps are used; all 24 practice questions are original.